POLYGLOT WORKSHOP

LLM PRETRAINING

PART I

Tutors
Nicholas Kluge & Aniket Sen

TRA SUSTAINABLE FUTURES

UNIVERSITÄT BONN

AGENDA

1. Frameworks and Parallelism

- ✓ Why bother with frameworks?
- ✓ Picking a training framework
- ✓ Parallelism Zoo (DDP, ZeRO, FSDP, HSDP, TP, SP, CP, PP …)

2. Architecture And Optimizations

- ✓ Implementing an Architecture
- ✓ Attention
- ✓ Saturation
- ✓ Optimization Techniques

3. Batch Size

- ✓ Finding an Efficient Batch Size
- ✓ Finding the Critical Batch Size

AGENDA

4. Training Stability

- ✓ Measuring Training Stability
- ✓ Keeping Things Stable
- ✓ Optimizers
- ✓ Learning Rate Scheduling
- ✓ The DeepSeek Heuristic

4. Unfortunately, Beyond the Scope

- ✓ Continual Pretraining
- ✓ Tokenizer Transplantation
- ✓ Checkpoint Management
- ✓ Pre-Marathon Checklist
- ✓ Final Thoughts

FRAMEWORKS AND PARALLELISM

Choosing Your Path

WHY BOTHER WITH FRAMEWORKS

- ✓ There is a myth in the community **(usually spread by people who don't actually do research or development)** that using training frameworks is cheating. “**Real developers implement stuff by themselves.”**

- ✓ But modern R&D **does not work like that**. Many **things that are incredibly tricky to implement** (e.g., PPO, Autograd, efficient communication primitives) have, **thankfully, already been done**. Frameworks exist precisely because the fundamentals are already well understood, and **endlessly reinventing them is a poor use of time** (but **supporting open-source projects is an awesome thing to do!**).

WHY BOTHER WITH FRAMEWORKS

With that said, it is important also to acknowledge **that implementing things has a lot of educational value**, and is definitely a road you should pursue if:

- ✔ You have **time to spare**.
- ✔ You want to develop a **better understanding of how such frameworks are built**.
- ✔ You want **complete control** of your training setup.

PICKING A TRAINING FRAMEWORK

The first decision we need to make is **which framework to use for training our model**. This choice involves balancing **three key considerations**:

- ✔ The framework must **support our target architecture** or allow us to extend it easily.
- ✔ It needs to be **stable, production-ready**, and **widely supported/used**.
- ✔ It should deliver **strong throughput** so we make the most of our compute budget (big MFUs!).

PICKING A TRAINING FRAMEWORK

Framework	Framework	Battle-Tested	Optimized	Lines of Code (core / total)	Extensibility & Debugging
Megatron-LM	Extensive	Kimi-K2, Nemotron	Pioneers of 3D parallelism	93k / 269k	Hard for beginners
DeepSpeed	Extensive	BLOOM, GLM	Pioneers of ZeRO & 3D parallelism	94k / 194k	Hard for beginners
TorchTitan	Growing feature set	Newer but tested by PyTorch team	Optimized for dense models, MoE underway	7k / 9k	Moderate: requires parallelism know-how
Nanotron	Minimal, tailored for HF pretraining	Yes (StarCoder, SmolLM)	Optimized to work with the HF ecosystem	15k / 66k	Moderate: requires parallelism know-how
Raw PyTorch	Barebones but powerful (DDP, FSDP, HSDP)	Core of all PyTorch frameworks	Simple, direct control over all primitives	— (user-defined)	Very easy to debug & extend manually
Accelerate	Simple multi-GPU abstraction (DDP/FSDP)	Used in Hugging Face ecosystem	Lightweight, integrates seamlessly with HF	~8k / 10k	Very easy; designed for minimal boilerplate

PICKING A TRAINING FRAMEWORK

- ✓ **Are you going to train a huge model (100B+ parameters)?** If so, consider Megatron-LM or DeepSpeed, as they offer mature support for advanced parallelism techniques such as 3D parallelism and ZeRO optimizations.

- ✓ **Do you want something more lightweight and easier to use?** TorchTitan or Nanotron are good options.

- ✓ **Do you want something even easier to use?** Use Accelerate.

- ✓ **Do you want to have complete control over every aspect of training?** PyTorch with DDP/FSDP/HSDP is the way to go, especially if you are comfortable implementing your own parallelism strategies. FSDP with a full shard is equivalent to DeepSpeed ZeRO3.

PICKING A TRAINING FRAMEWORK

ADVISE: While Megatron-LM can be challenging for beginners, recent efforts have improved its usability, such as the NeMo Framework, an **end-to-end training framework** for large language models and more. It **enables seamless scaling** of training (both pretraining and post-training) workloads from **single GPU to thousand-node clusters** for both **Hugging Face/PyTorch and Megatron** models. The fact that it supports Hugging Face models means you can easily switch between Megatron and other frameworks. Meanwhile, the implementation of training loops, data loading, and checkpointing is all **much more user-friendly than raw Megatron**.

PARALLELISM ZOO

Training a modern LLM is less ***“one giant brain thinking really hard”*** and more ***“a perfectly choreographed flash mob of GPUs.”*** Parallelism is how we split the work so models with billions (or trillions) of parameters can actually learn.

- ✔ Distributed Data Parallel (DDP)
- ✔ ZeRO / FSDP
- ✔ Tensor Parallelism (TP)
- ✔ Sequence Parallelism (SP)
- ✔ Context Parallelism (CP)
- ✔ Pipeline Parallelism (PP)

Together, these strategies let us turn a **single impossible training job** into a **carefully synchronized, massively parallel performance**.

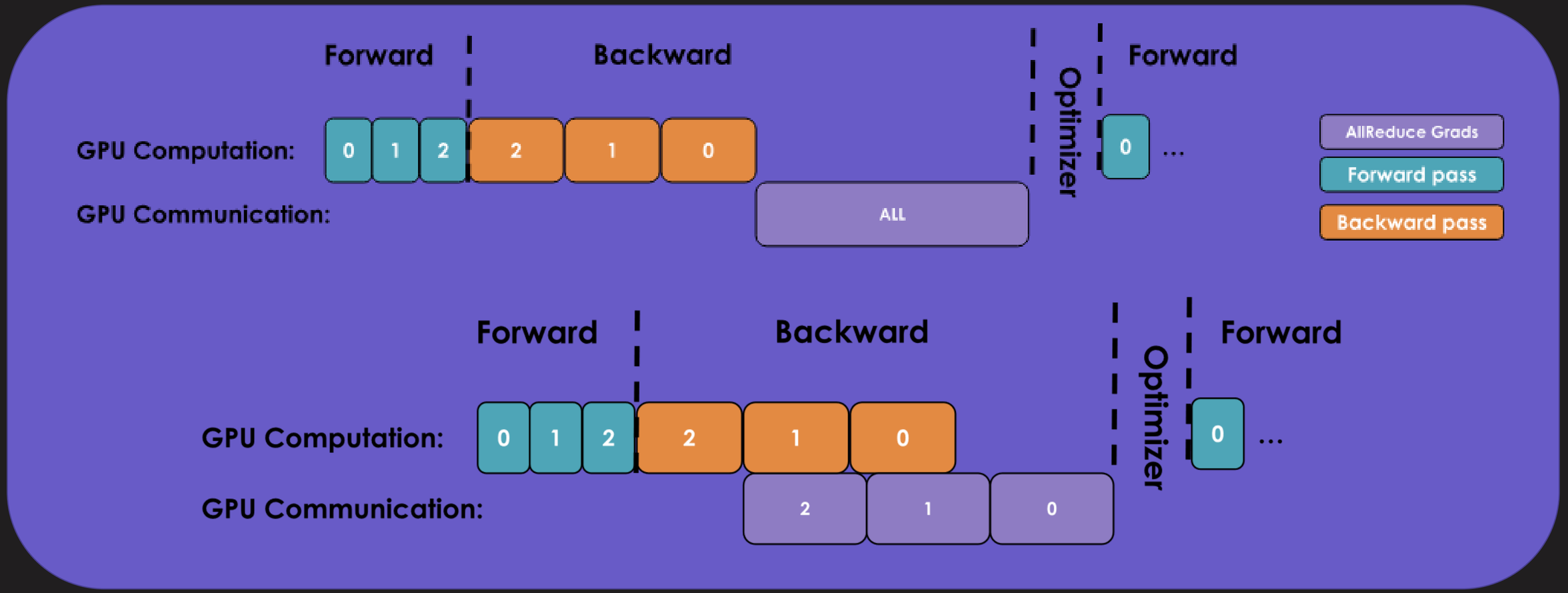
PARALLELISM ZOO - DDP

- ✓ The idea behind data parallelism (DP or DDP) is to **replicate the model on several GPUs and run forward and backward passes on different micro-batches of data in parallel on each GPU**. Distributed communication occurs via gradients (AllReduce), which effectively allows us to scale the batch size to very large values, and **transformers prefer large batch sizes**.

- ✓ Remember that **AllReduce** is one of the communication methods that are **less penalized as we scale with replicas**. Hence, we should always try to **scale it!**

>> **torch.nn.parallel.DistributedDataParallel**

PARALLELISM ZOO - DDP



***Overlapping** the backward pass with AllReduce (**bucketing gradients into chunks**) **hides communication overhead within the computation**.*

Source → The Ultra-Scale Playbook

PARALLELISM ZOO – ZERO / FSDP

- ✓ When a full batch fits on a single GPU, the best approach is almost always to **scale with DDP**. In practice, this means most small-to-mid-scale models (≈ <3 B parameters) can be trained comfortably in this regime. However, there are cases where even a single sample (batch size = 1) **does not fit in GPU memory**, and that's exactly where the other parallelism strategies come into play.

- ✓ As we move beyond DDP, our next stop—both in terms of added complexity and potential gains—should, in my view, be **ZeRO / FSDP**.

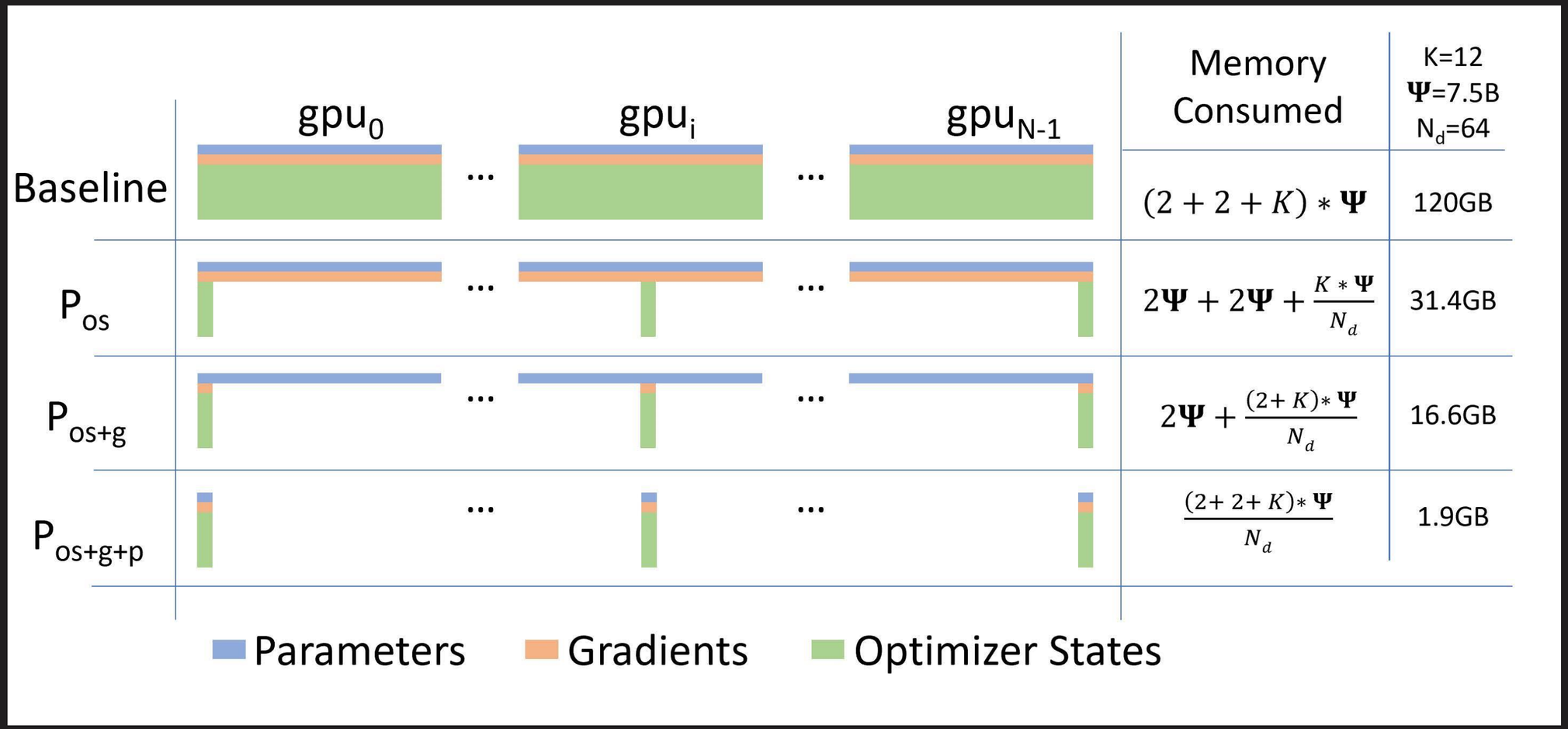
>> **torch.distributed.fsdp.fully_shard** **(FSDP v2!)**

PARALLELISM ZOO – ZERO / FSDP

- ✓ ZeRO is a memory optimization technique that **reduces memory redundancy during training large neural networks (🧶).** In DDP, we replicate the optimizer state across replicas, which is **redundant (the same applies to gradients and, ultimately**, parameters). ZeRO eliminates this by **partitioning the optimizer state, gradients, and parameters across the data-parallel dimension** (it is still **1D parallelism**!).

- ✓ How much space does everything take?
  - ✔ Model's **parameters** (half precision; i.e., BF16/FP16): **2B (Bytes)**
  - ✔ Model's **gradients** (half precision; i.e., BF16/FP16): **2B**
  - ✔ Model's **parameters in FP32** and **optimizer states**: **4B + (4B + 4B)**

PARALLELISM ZOO – ZERO / FSDP



Source → ZeRO: Memory Optimizations Toward Training Trillion Parameter Models

PARALLELISM ZOO – ZERO / FSDP

- ✓ **Stage 1 →** Only optimizer states are sharded. **Not recommended, because it is almost always better to use Stage 2**. Also, ***no one*** in PyTorch is working on maintaining ZeroRedundancyOptimizer.

- ✓ **Stage 2 →** Since on **each replica we only need to have the gradient shard corresponding to its optimizer state shard**, it makes sense to shard gradients as well, similarly to the optimizer states. Then, during the backward pass, **instead of performing an all-reduce over the gradients**, we **only perform a reduce-scatter operation!**

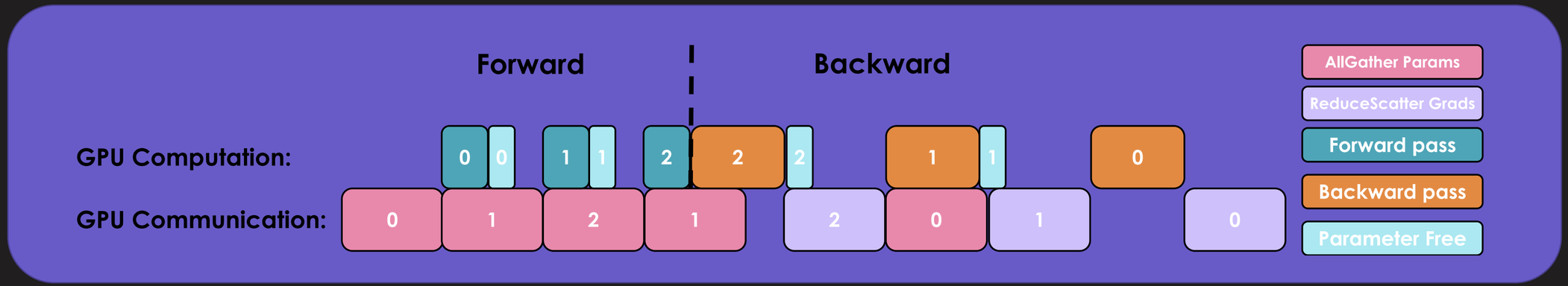
- ✓ **Let us see how this looks, step-by-step!**

PARALLELISM ZOO – ZERO / FSDP

- ✓ **Forward pass →** each replica uses the full set of BF16 parameters, but processes different micro-batches (**DDP**).
- ✓ **Backward pass →** each replica **computes gradients from its own micro-batches**.
- ✓ reduce-scatter **→ Gather gradients from all replicas**, average them, and scatter the output in equal-sized blocks between ranks.
- ✓ **Optimizer Step →** Each replica updates its **local optimizer states** to produce chunks of updated FP32 parameters, then casts them to BF16.
- ✓ **Model Update →** An all-gather is performed on the BF16 parameters **so every replica reconstructs the full parameter set**.

PARALLELISM ZOO – ZERO / FSDP

- ✓ **Stage 3 / FSDP →** As we **perform the forward pass** and sequentially go through the layers, **we retrieve the necessary parameters on demand and immediately flush them from memory when we don't need them anymore**. The **backward pass** works the same way, just **inverted in flow**.
- ✓ We need to do these **all-gathers continuously throughout the forward and backward pass in a training step**, which amounts to **2 X num_layers – 1** additional all-gathers in a training step compared to ZeRO-2. **Each incurs a small base latency overhead, but prefetching enables overlap between computation and communication.**



Source → The Ultra-Scale Playbook

How FSDP Works? All-Gather

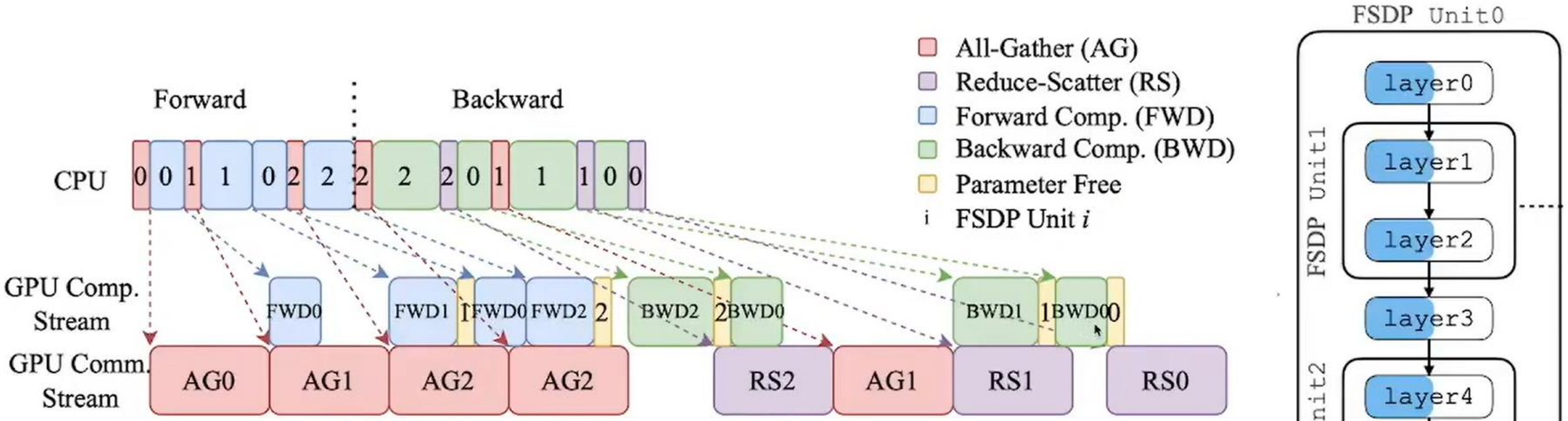


Figure 5: Overlap Communication and Computation

PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel https://arxiv.org/abs/2304.11277

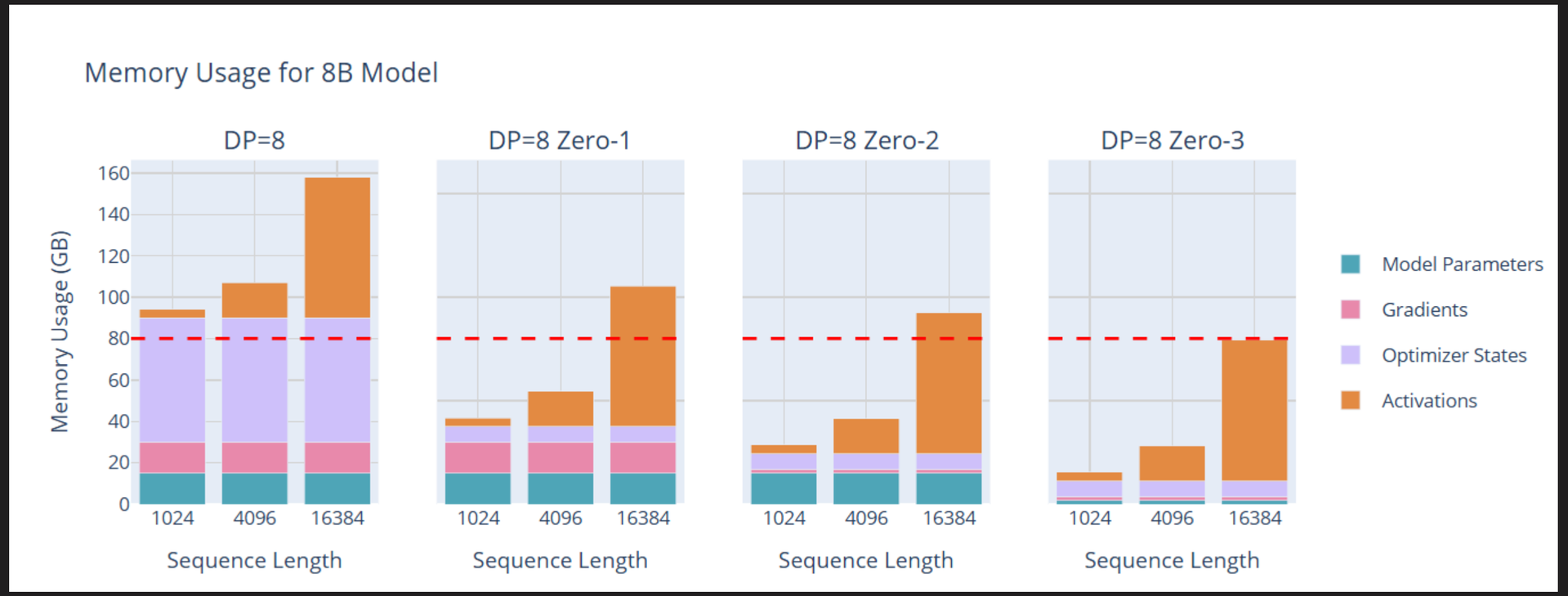
The Best explanation of FSDP to date: *"How Fully Sharded Data Parallel (FSDP) works?"*, by Ahmed Taha

PARALLELISM ZOO – ZERO / FSDP

QUESTION: *What is the thing that **ZeRO 3 / FSDP is not partitioning?***

PARALLELISM ZOO – ZERO / FSDP

QUESTION: *What is the thing that **ZeRO 3 / FSDP is not partitioning?***



Source → The Ultra-Scale Playbook

PARALLELISM ZOO – HSDP

The strategy:

- ✓ Cross-host communication is generally **slower and more expensive**.
- ✓ FSDP requires **more complex and communication-heavy operations**, so restricting FSDP to the high-bandwidth intra-host environment is more efficient.
- ✓ DDP across hosts ensures global synchronization without **incurring the full communication cost of FSDP** inter-node.

💡 How to use DeviceMesh with HSDP?

```
# Hybrid Sharding Data Parallel (HSDP)
#
# - Across hosts: DDP
# - Within a host: FSDP
#
# Example: 2 hosts (nodes), each with 4 GPUs
=============================================
-------------       -------------
|   Host 1  |       |   Host 2  |
|  4 GPUs   |       |  4 GPUs   |
|           |       |           |
|  (FSDP)   |  ...  |  (FSDP)   |
| [0,1,2,3] |       | [4,5,6,7] |
|           |       |           |
-------------       -------------
DDP groups across hosts:
[0, ..., 3], [4, ..., 7]
```

PARALLELISM ZOO - TP

- ✓ Tensor Parallelism (TP) shards tensors across GPUs along a chosen dimension, distributing **parameters, gradients, optimizer states, and activations** without ever fully replicating the model on a single device. Layers can be sharded either **column-wise** (broadcast inputs, all-gather outputs) or **row-wise** (scatter inputs, all-reduce outputs), each requiring different communication primitives but enabling **scalable matrix multiplication**.

- ✓ Doing TP requires you to know how to slice MLPs (**column-parallel layer followed by a row-parallel layer**) and attention blocks (**Q/K/V projections are column-parallel, output projection is row-parallel**), which is significantly **more complex than setting up ZeRO 3 / FSDP**.

- ✓ Operations like LayerNorm and dropout still require full activations (**full hidden dimension to compute mean and variance**), limiting memory savings. Also, **TP scales poorly when performed across hosts/nodes**.

>> **torch.distributed.tensor.parallel**

PARALLELISM ZOO - SP

- ✓ **Sequence Parallelism (SP)** reduces memory usage by **sharding the input sequence dimension across GPUs**, thereby distributing activation storage for long sequences. It is tightly coupled with tensor parallelism and is primarily applied to **dropout and layer normalization**, where activations can be partitioned along the sequence dimension.

- ✓ Here, sequence parallelism does **not** address the attention-scaling bottleneck. When attention over long contexts becomes dominant, techniques such as **Ring Attention** are required; these are better described as **context parallelism** rather than sequence parallelism in the TP sense.

- ✓ Like vanilla TP, **TP+SP introduces communication on the critical path** that is difficult to overlap with computation, making performance highly dependent on interconnect bandwidth. As a result, TP+SP is typically confined to **within-node parallelism**.

PARALLELISM ZOO - CP

- ✓ **Context Parallelism (CP)** extends sequence parallelism by distributing **the sequence dimension across GPUs for the entire model**, including modules that are already tensor-parallel. Sharding along both the **model (TP)** and **sequence (CP)** dimensions, CP significantly reduces activation memory growth with sequence length.

- ✓ The main challenge arises in the **attention blocks**, where each token must attend to keys and values from all other tokens. Since the sequence is distributed across GPUs, this requires exchanging K/V data between devices (Ring Attention)

- ✓ There are two main communication strategies for attention under CP: all-gather, in which each GPU collects all K/V pairs at once, and **ring** all-to-all, where K/V chunks are exchanged progressively (high communication complexity).

PARALLELISM ZOO - PP

- ✓ **Pipeline Parallelism (PP)** splits a model into **sequential stages**, assigning each stage to a different GPU. **Each GPU processes its micro-batch and passes intermediate activations to the next stage**.

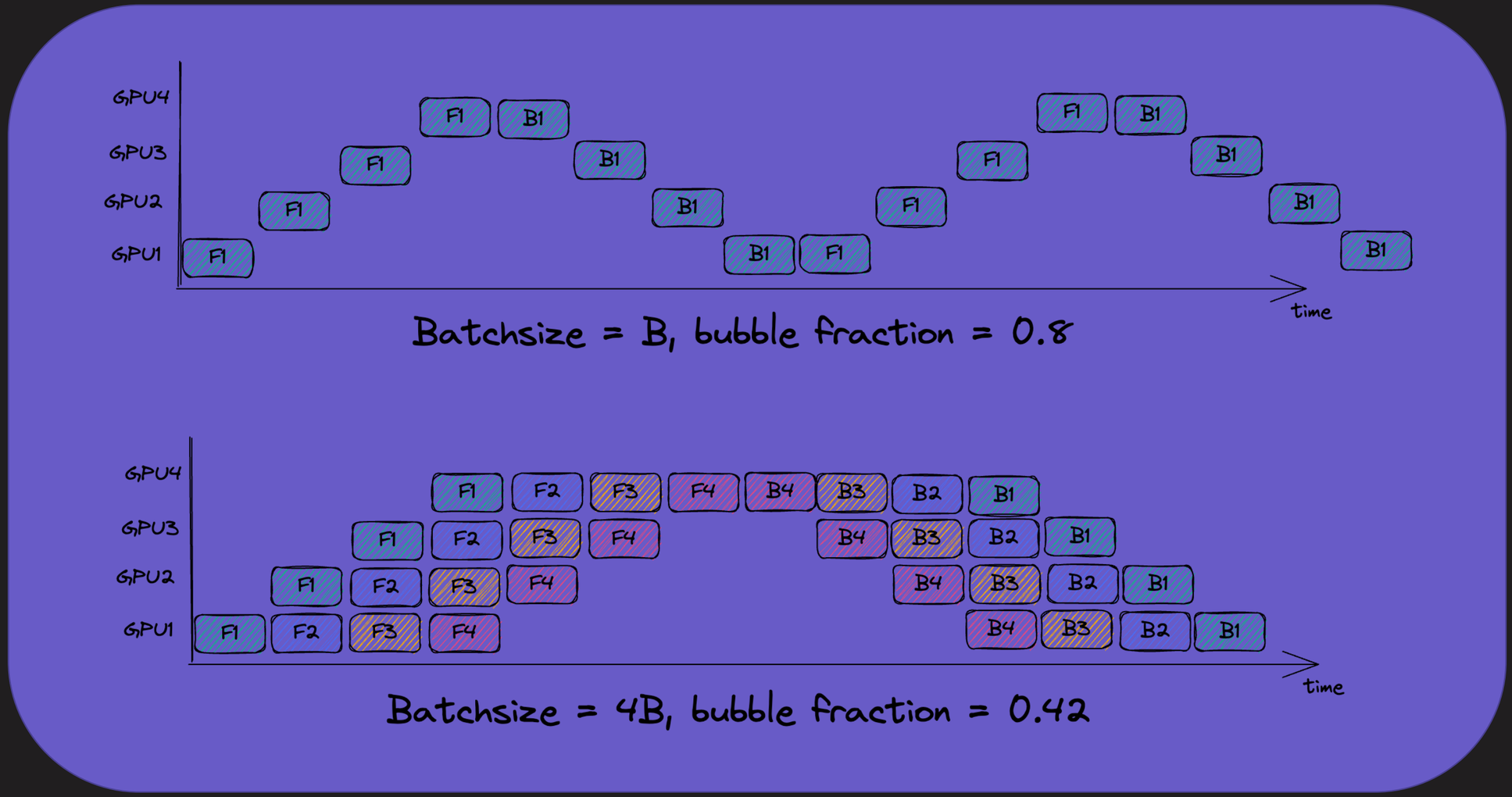
💡 **QUESTION:** ***Do you see the limitation?***

PARALLELISM ZOO - PP

To improve efficiency, micro-batches can be used to **overlap computation across stages**, reducing idle time (the ***"bubble"***).

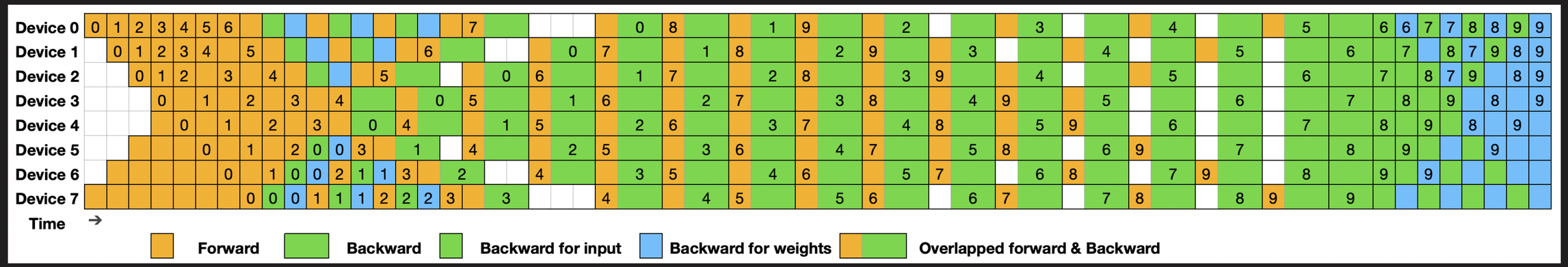
- ✔ **All Forward All Backward (📎)**
- ✔ **1 Forward, 1 backward (📎)**
- ✔ **Zero Bubble (📎)**
- ✔ **DualPipe (📎)**

PARALLELISM ZOO - PP



Simon Boehm. (2022). *Pipeline-Parallelism: Distributed Training via Model Partitioning*.

PARALLELISM ZOO - PP



DualPipe is an innovative bidirectional pipeline parallelism algorithm introduced in the *DeepSeek-V3 Technical Report*.

PARALLELISM ZOO

Method	Memory savings apply specifically on…	Parallel/sharding dimension	Disadvantage
DDP	Activations (reduce local batch size)	Batch	Limited by max batch size
PP	Model Parameters	Model Layers	Idle bubble and complex schedules
TP+SP	Model Parameters and Activations	Hidden Dimension/Sequence Length	Requires high-bandwidth communication
CP	Activations	Sequence Length	Adds communication overhead in attention modules
ZeRO-1	Optimizer States	Sharded among DP Replicas	Communication overhead, does not affect activations
ZeRO-2	Optimizer States and Gradients	Sharded among DP Replicas	Communication overhead, does not affect activations
ZeRO-3	Optimizer States, Gradients, and Model Parameters	Sharded among DP Replicas	Communication overhead, does not affect activations

Source → The Ultra-Scale Playbook

BREAK!

ARCHITECTURE AND OPTIMIZATIONS

Tricks to Make it Fit

FINDING A STARTING ARCHITECTURE

Every successful model stands on the shoulders of earlier ones. **Qwen** started from **Llama**. **Llama 3** started from **Llama 2**. **Kimi K2** builds on **DeepSeek-V3's MoE design**. This pattern isn't copying — it's **evolution**. Good architectures and training setups take years of collective trial and error and refinement across many teams. A ***"proven foundation"*** quietly bundles all of this hard-won knowledge. Starting from scratch means ***rediscovering every sharp edge yourself***.

Innovation still happens — but mostly in small, careful steps. One better attention mechanism. One smarter routing rule. One more stable normalization. Over time, these **small deltas accumulate into major advances**, even though each individual change looks modest.

💡 Always start from battle-tested foundations. ***Especially if this is your debut in the LLM arena.***

FINDING A STARTING ARCHITECTURE

If you look at recent models like **Qwen3**, **Gemma3**, or **DeepSeek v3**, you'll see that despite their differences, they **all share the same foundation** — the transformer architecture introduced in 2017. What's changed over the years isn't the fundamental structure, **but the refinements to its core components (transformer++)**.

- ✔ **RMSnorm** for normalization (📎**).**
- ✔ **RoPE** positional embeddings (📎**)**
- ✔ **SwiGLU/SiLU** activations (📎**)**
- ✔ New ways to do **Attention (MQA, GQA, MLA, GA).**

💡 Llama, Qwen, Falcon, GPT-OSS, and DepSeek are all examples of transformer++ architectures. ***Mambas, MoEs, and Hybrid architectures will be outside the scope of this workshop***.

IMPLEMENTING AN ARCHITECTURE

- ✓ In terms of implementation, the **Hugging Face Transformers library** has become a popular choice for implementing transformer architectures, **providing a wide range of model types and configurations for designing custom models**.
- ✓ It is a no-brainer to use it unless you have very specific needs that **require a very custom/experimental implementation**.
- ✓ Using them also **guarantees compatibility** with the wider ecosystem of tools and libraries built around Hugging Face.

```python
from transformers import AutoConfig, AutoModelForCausalLM

# Choose a reference architecture (Llama, but could be anything!)
REFERENCE_MODEL = "HuggingFaceTB/SmolLM2-135M"

# Minimal set of architectural knobs to play with
config = AutoConfig.from_pretrained(
    REFERENCE_MODEL,
    # Core model dimensions
    hidden_size=512,
    intermediate_size=2048,
    num_hidden_layers=12,
    num_attention_heads=8,
    vocab_size=32000,
    max_position_embeddings=2048,
    tie_word_embeddings=True
)

# Instantiate model from scratch (randomly initialized)
model = AutoModelForCausalLM.from_config(config)
params = sum(p.numel() for p in model.parameters() if
print(model)
print(f"Number of trainable parameters: {params:,}")
```

IMPLEMENTING AN ARCHITECTURE

- ✓ In terms of implementation, the Hugging Face Transformers library has become a popular choice for implementing transformer architectures, **providing a wide range of model types and configurations for designing custom models**.
- ✓ It is a no-brainer to use it unless you have very specific needs that **require a very custom/experimental implementation**.
- ✓ Using them also **guarantees compatibility** with the wider ecosystem of tools and libraries built around Hugging Face.

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(32000, 512)
    (layers): ModuleList(
      (0-11): 12 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=512, out_features=512, bias=False)
          (k_proj): Linear(in_features=512, out_features=192, bias=False)
          (v_proj): Linear(in_features=512, out_features=192, bias=False)
          (o_proj): Linear(in_features=512, out_features=512, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=512, out_features=2048, bias=False)
          (up_proj): Linear(in_features=512, out_features=2048, bias=False)
          (down_proj): Linear(in_features=2048, out_features=512, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((512,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((512,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((512,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=512, out_features=32000, bias=False)
)
Number of trainable parameters: 62,796,288
```

IMPLEMENTING AN ARCHITECTURE

- ✓ In terms of implementation, the Hugging Face Transformers library has become a popular choice for implementing transformer architectures, **providing a wide range of model types and configurations for designing custom models**.
- ✓ It is a no-brainer to use it unless you have very specific needs that **require a very custom/experimental implementation**.
- ✓ Using them also **guarantees compatibility** with the wider ecosystem of tools and libraries built around Hugging Face.

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(32000, 512)
    (layers): ModuleList(
      (0-11): 12 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=512, out_features=512, bias=False)
          (k_proj): Linear(in_features=512, out_features=192, bias=False)
          (v_proj): Linear(in_features=512, out_features=192, bias=False)
          (o_proj): Linear(in_features=512, out_features=512, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=512, out_features=2048, bias=False)
          (up_proj): Linear(in_features=512, out_features=2048, bias=False)
          (down_proj): Linear(in_features=2048, out_features=512, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((512,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((512,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((512,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=512, out_features=32000, bias=False)
)
Number of trainable parameters: 62,796,288
```

IMPLEMENTING AN ARCHITECTURE

Core architectural hyperparameters:

- ✔ **Vocabulary Size →** Dimensionality of the input embedding matrix and the output logits layer.
- ✔ **Hidden Size →** Dimensionality of the residual stream. All layers read from and write to this vector space.
- ✔ **Number of Hidden Layers →** Depth of the transformer stack
- ✔ **Number of Attention Heads →** Number of parallel attention mechanisms per layer.
- ✔ **Number of Key/Value Heads → MQA / GQA**.
- ✔ **Intermediate Size (MLP Dimension) →** Width of the feed-forward network inside each layer.
- ✔ **Maximum Position Embeddings →** context window.

💡 For **efficient GPU utilization**, pay special attention to:

>> hidden_size, max_position_embeddings, and batch_size

>> These dimensions should be chosen so they **align well with GPU** hardware (e.g., **multiples of 64 or 128**).

IMPLEMENTING AN ARCHITECTURE

- ✓This configuration is an example of a model configuration file that specifies the dimensions of LilTii and LilMoo. These values were tuned to **maximize performance while fitting into 8 x NVIDIA A100-SXM4-80GB GPUs**. In other words, they were tuned to fit a specific deployment target.
- ✓These were modified from the original Qwen3-0.6B architecture.

```
# LilTii / LilMoo
{
  "head_dim": 96,
  "hidden_size": 1536,
  "intermediate_size": 3072,
  "max_position_embeddings": 4096,
  "num_attention_heads": 16,
  "num_hidden_layers": 28,
  "num_key_value_heads": 8,
  "tie_word_embeddings": true,
  "vocab_size": 49152
}
```

```
# Qwen3-0.6B-Base
{
  "head_dim": 128,
  "hidden_size": 1024,
  "intermediate_size": 3072,
  "max_position_embeddings": 4096,
  "num_attention_heads": 16,
  "num_hidden_layers": 28,
  "num_key_value_heads": 8,
  "tie_word_embeddings": true,
  "vocab_size": 151936
}
```

DEEP-AND-THIN VS SHALLOW-AND-WIDE

- ✓ At a broader scale, architectural layout choices also shape model behavior. Although the total parameter count largely determines a language model's capacity, **how those parameters are distributed across depth and width also matters**.

- ✓ Deeper models outperform equally sized, wider ones on language modeling and compositional tasks until the benefit plateaus**. This "deep-and-thin" strategy works well for small LLMs (MobileLLM, ModernBERT, Smollm2, Smollm3, DeepSeek 67B).**

- ✓ **Wider models tend to offer faster inference thanks to greater parallelism.**

ATTENTION

- ✓ At a broader scale, architectural layout choices also shape model behavior. Although the total parameter count largely determines a language model's capacity, **how those parameters are distributed across depth and width also matters**.

- ✓ Deeper models outperform equally sized, wider ones on language modeling and compositional tasks until the benefit plateaus**. This "deep-and-thin" strategy works well for small LLMs (MobileLLM, ModernBERT, Smollm2, Smollm3, DeepSeek 67B).**

- ✓ **Wider models tend to offer faster inference thanks to greater parallelism.**

ATTENTION

- ✓ In the standard attention implementation, the main idea is that you have **N** attention heads each **independently doing the same retrieval task**: transform the hidden state into **queries**, **keys**, and **values**, then use the current query to retrieve the **most relevant token (What is attention?)**.

- ✓ At inference time, **we don't need to recompute the KV values for past tokens and can reuse them**. The memory for past KV values is called the KV-Cache. As context windows grow, this cache can **quickly become an inference bottleneck** and consume a large share of GPU memory (What is the KV-Cache?).

- ✓ To reduce the size of this cache, there are a lot of modified attention mechanisms **that tweak and change how KV values operate across the attention heads.**

```
n_bytes = 2    # Bytes per element (e.g., 2 for fp16)
seq = 2048     # Sequence length
n_layers = 24  # Number of transformer layers
n_heads = 16   # Number of attention heads
d_head = 64    # Dimension per head
n_kv_heads = 4 # Number of key/value heads (for GQA)


# Multi-Head Attention (MHA)
mha_cache = 2 * n_bytes * seq * n_layers * n_heads * d_head
# Multi-Query Attention (MQA)
mqa_cache = 2 * n_bytes * seq * n_layers * 1 * d_head
# Grouped-Query Attention (GQA)
gqa_cache = 2 * n_bytes * seq * n_layers * n_kv_heads * d_head
# MMulti-head Latent Attention (MLA)
mla_cache = 4.5 * n_bytes * seq * n_layers * d_head

print(f"MHA KV-cache: {mha_cache / 1e9:2f} GB")  # 0.201327 GB
print(f"MQA KV-cache: {mqa_cache / 1e9:2f} GB")  # 0.012583 GB
print(f"GQA KV-cache: {gqa_cache / 1e9:2f} GB")  # 0.050332 GB
print(f"MLA KV-cache: {mla_cache / 1e9:2f} GB")  # 0.028312 GB
```

ATTENTION

- ✓ Results from the ablations of SmolLm3 (🧶) indicate that **GQA with 2, 4, or 8 groups can match MHA performance, while MQA underperforms significantly**.

- ✓ The MLA vs GQA ablation for SmolLm3 (🧶) indicates that **GQA outperforms MLA**. In the end, the choice between GQA and MLA may depend on an evaluation that you yourself will have to run for your specific use case.

```
n_bytes = 2    # Bytes per element (e.g., 2 for fp16)
seq = 2048     # Sequence length
n_layers = 24  # Number of transformer layers
n_heads = 16   # Number of attention heads
d_head = 64    # Dimension per head
n_kv_heads = 4 # Number of key/value heads (for GQA)


# Multi-Head Attention (MHA)
mha_cache = 2 * n_bytes * seq * n_layers * n_heads * d_head
# Multi-Query Attention (MQA)
mqa_cache = 2 * n_bytes * seq * n_layers * 1 * d_head
# Grouped-Query Attention (GQA)
gqa_cache = 2 * n_bytes * seq * n_layers * n_kv_heads * d_head
# MMulti-head Latent Attention (MLA)
mla_cache = 4.5 * n_bytes * seq * n_layers * d_head

print(f"MHA KV-cache: {mha_cache / 1e9:2f} GB")  # 0.201327 GB
print(f"MQA KV-cache: {mqa_cache / 1e9:2f} GB")  # 0.012583 GB
print(f"GQA KV-cache: {gqa_cache / 1e9:2f} GB")  # 0.050332 GB
print(f"MLA KV-cache: {mla_cache / 1e9:2f} GB")  # 0.028312 GB
```

MUCH MORE ...

Things outside our scope due to **time reasons:**

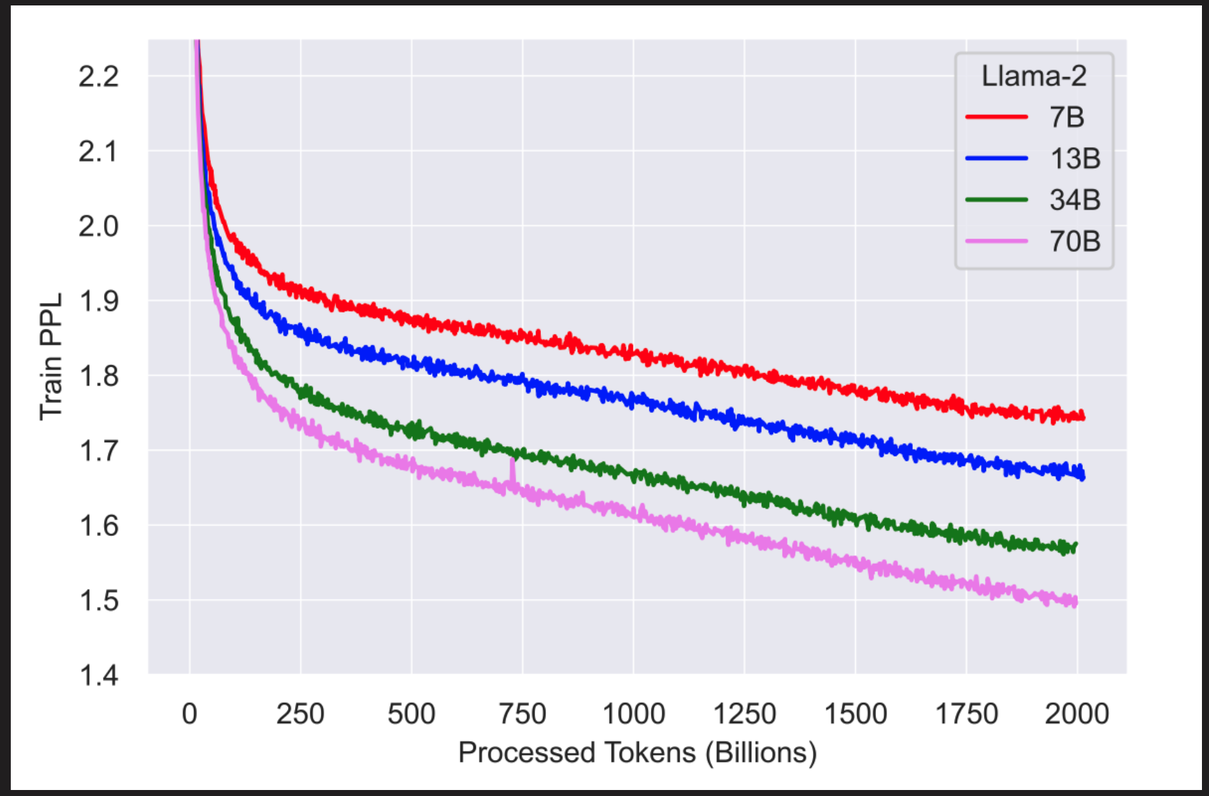
- ✔ **Chunked Attention (📎)**
- ✔ **Sliding Window Attention (SWA) (📎)**
- ✔ **Attention Masking (📎)**
- ✔ **Attention Sinks (📎, but Gated Attention apparently solved this)**
- ✔ **Context/RoPE Extentsion (RoPE ABF:📎, YaRN: 📎, NoPE: 📎)**

💡 I encourage you all to explore and research while you still can. Modern deep learning and LLMs are still accessible enough that it's possible to grasp the theory **without growing old**.

💡 **Must READ:** The Big LLM Architecture Comparison, by Sebastian Raschka.

SATURATION AND TARGETING A MODEL SIZE

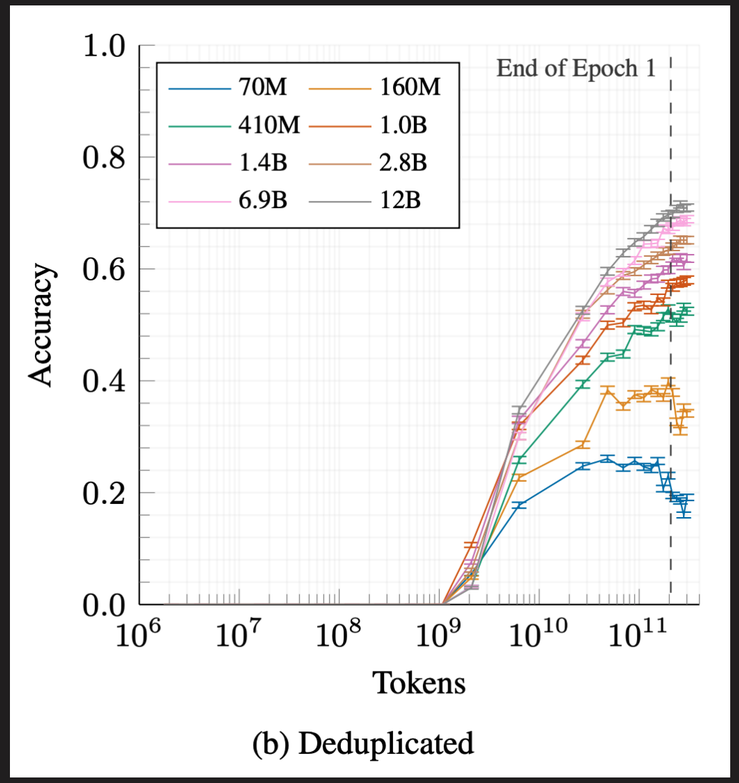
- ✓ *What is saturation?* **Saturation** refers to the point at which a model stops improving meaningfully on its task, even as it continues to see more training data. For example, in the Llama 2 paper, the authors note that even after pretraining on 2 trillion tokens (beyond the Chinchilla 20 tokens/param), the model's training loss continues to decrease, indicating it has **not yet saturated**—more data could still improve performance.



Source: Llama 2: Open Foundation and Fine-Tuned Chat Models.

SATURATION AND TARGETING A MODEL SIZE

- ✓ Similarly, the Pythia results show that smaller models (70M and 160M parameters) reach saturation in LAMBADA accuracy with sufficient tokens, whereas larger models (410M and above) continue improving and do not saturate under **the same training regime**.



Source: Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling.

OPTIMIZATION TECHNIQUES

We have our model—its architecture is set, and the number of parameters fixed. Now comes the long list of tricks and hacks to cram the largest possible batch into the GPU. **Our sworn enemy: CUDA OOM**.

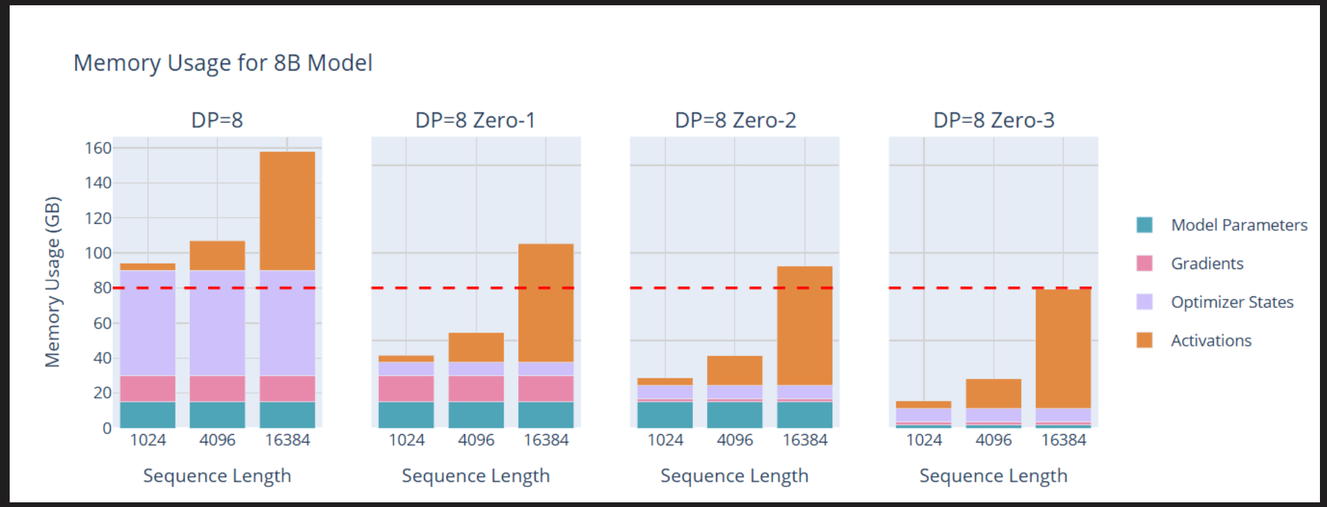
Batch size is one of the hyperparameters with the most **direct impact on memory usage**. A larger batch processes more data in parallel, improving GPU utilization, but it also increases memory demand for storing activations and gradients. **For recent LLM training, the sweet spot usually lies between 2 million and 60 million tokens per batch.**

- ✔ **Smollm3 →** ~2M tokens → for 11.2 trillion tokens (📎)
- ✔ **Llama 1 →** ~4M tokens → for 1.4 trillion tokens (📎)
- ✔ **DeepSeek →** ~60M tokens → for 14 trillion tokens (📎)

What should we do when we **don’t have enough memory** to achieve our target batch size?

ACTIVATION RECOMPUTATION / RE-MATERIALIZATION

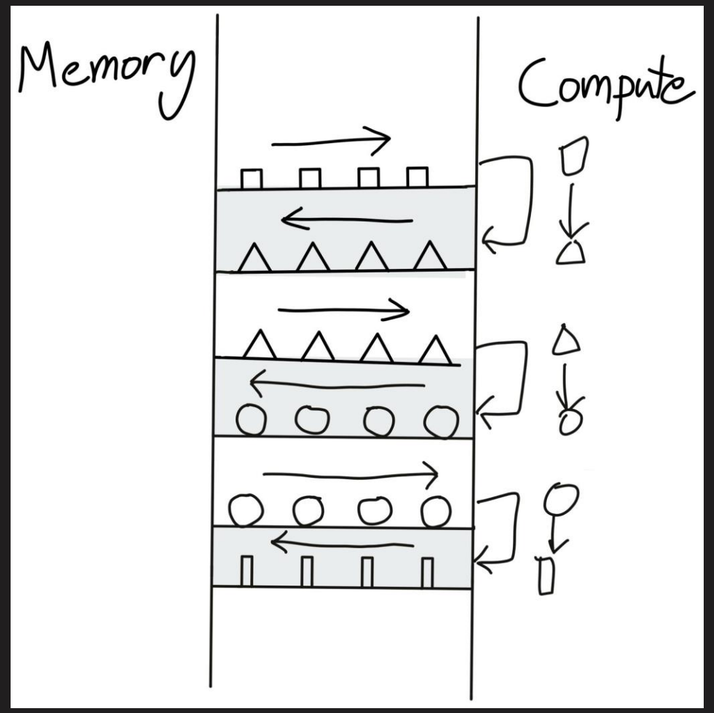
- ✓ Activation recomputation is a memory-saving technique (Chen et al. 2016), where **some activations from the forward pass are discarded and recomputed during the backward pass**, trading extra computation for reduced memory use.
- ✓ Most training frameworks and transformer implementations provide a simple API to set this feature.

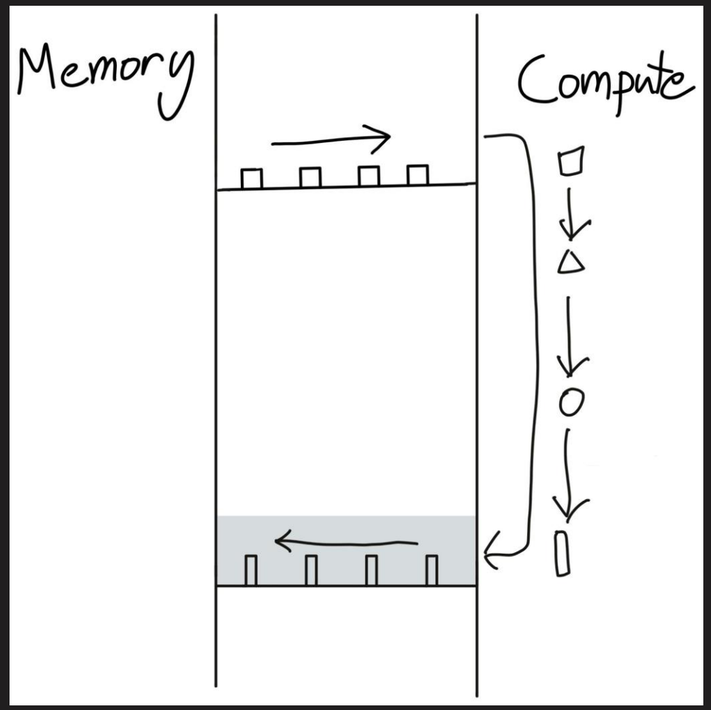


Source → The Ultra-Scale Playbook

FUSED KERNELS

- ✓ By now, it should be clear that most deep learning workloads are memory-bound—that is, limited by **how fast we can move data around**.
- ✓ Overlapping communication and computation (for example, using asynchronous CPU–GPU transfers) helps, but there is another guiding principle worth adopting: **avoid, at all costs, bouncing back and forth between the host and GPU kernel launches.**





Making Deep Learning Go Brrrr From First Principles, by Horace He.

FUSED KERNELS

- ✓ Fused kernels **pack as many successive compute operations as possible together in a single kernel** for the GPU to run.

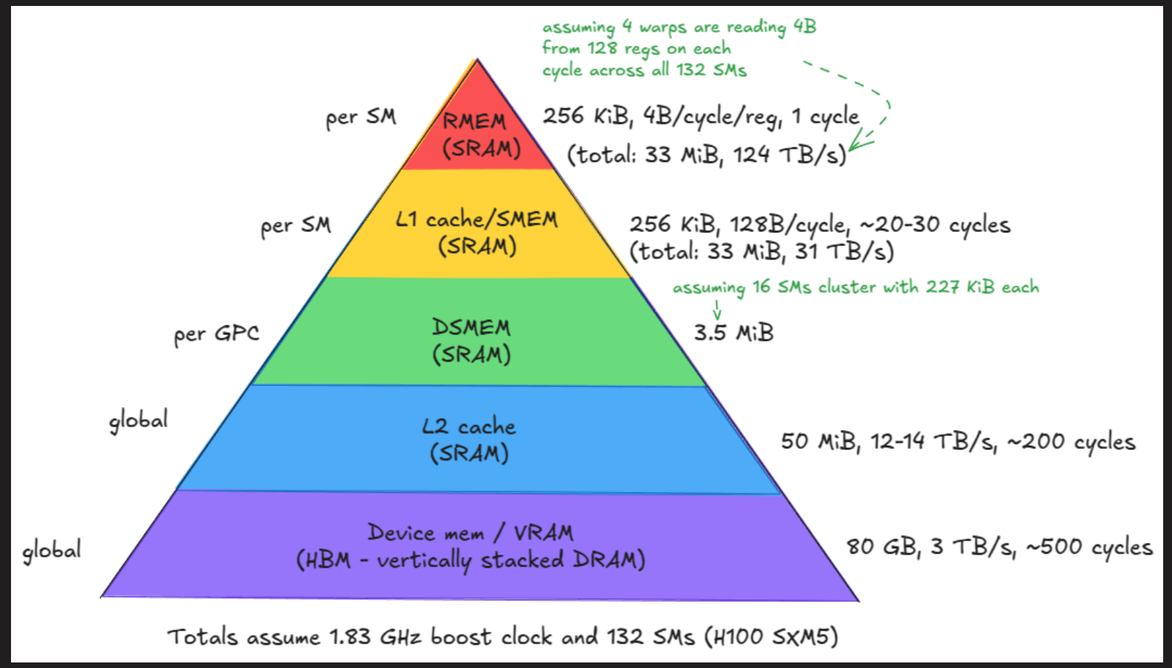
- ✓ Fused kernels are particularly efficient and straightforward to implement for **sequences of pointwise operations** that operate independently on each input token. In this case, **there is no point in sending the computed values back to global memory before moving them to SM memory and spinning up a new kernel**. It is more efficient to retain all values locally until all computations have been completed.

- ✓ In a Transformer model, this approach can be applied whenever we have a sequence of point-wise operations (e.g., **Norm layers**).

FLASHATTENTION

- ✓ FlashAttention was introduced by Tri Dao and proposed to optimize attention computations by writing custom CUDA kernels to make them much faster and more memory efficient.

- ✓ A basic implementation of the attention mechanism involves **a lot of transfer between memory and workers**. It requires materializing the **S** matrix (**where S = QK^T, the attention scores**) and the **P** matrix (**where P = softmax of S**) in HBM, which means the results must be sent to HBM and then back to SRAM for subsequent computations.

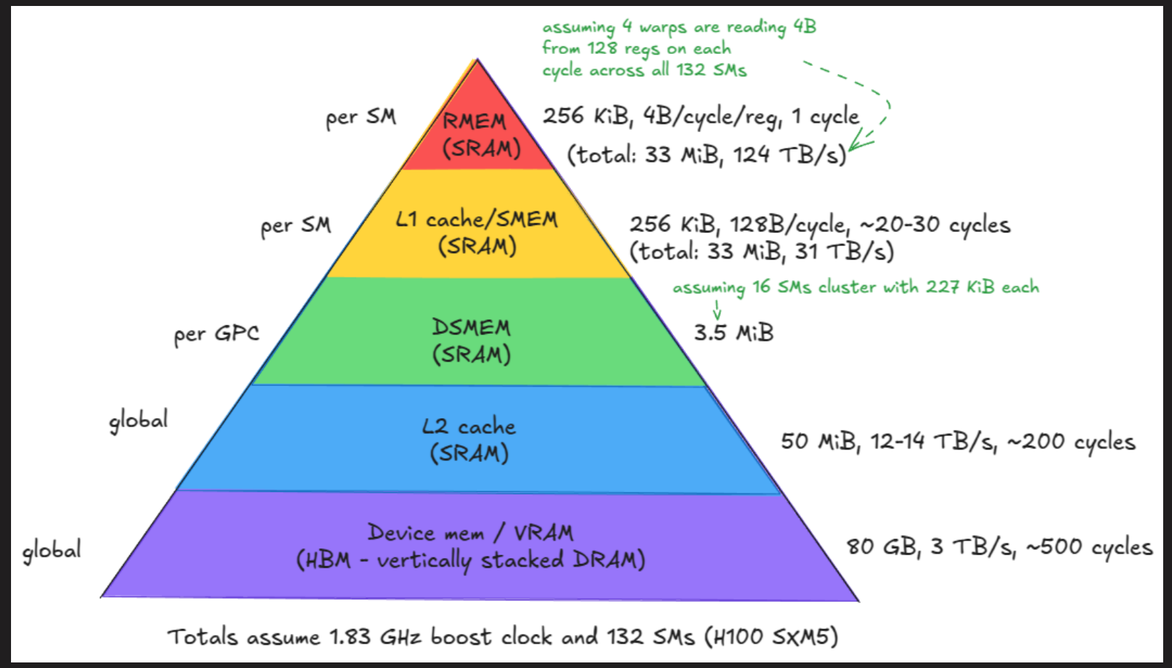


Source → Aleksa Gordić (MatMul blog)

FLASHATTENTION

- ✓ By **avoiding materializing the S matrix**, we reduce the memory burden of attention. We also remove a substantial portion of the O(S^2) attention cost. All variants of linear attention and subquadratic approaches to approximate attention have **largely been set aside in favor of the exact and fast FlashAttention implementation and mechanism**.

- ✓ FlashAttention-2 reaches up to 73% of the theoretical peak FLOPs/s on A100 GPUs (**almost as fast as GEMM's!**).



Source → Aleksa Gordić (MatMul blog)

LIGER KERNEL

- ✓ Liger Kernel is a collection of highly optimized Triton kernels specifically designed for LLM training. **These kernels fuse operations such as RMSNorm, RoPE, SwiGLU, and CrossEntropy, enabling up to 20% higher multi-GPU throughput and a 60% reduction in memory footprint during training**.

- ✓ These kernels are lightweight (minimal dependencies) and **compatible with all PyTorch native frameworks** (DDP, FSDP, DeepSpeed, Accelerate).

💡 **Must watch:** *Byron Hsu's presentation on the Liger-Kernel*

PYTORCH'S TURBO BUTTON

- ✓ `torch.compile` → PyTorch's just-in-time compiler that turns your eager PyTorch model into an optimized version that runs faster and often uses memory more efficiently.

⚡ **Speedups with almost no code changes**

🧠 **All about kernel fusion!**

Unfortunately, it breaks when you combine it with other stuff …

```python
import torch, time

model = torch.nn.Sequential(
    torch.nn.Linear(4096, 4096),
    torch.nn.ReLU(),
    torch.nn.Linear(4096, 4096)
).cuda()

x = torch.randn(1024, 4096, device="cuda")

# Warmup
for _ in range(10):
    model(x)

compiled = torch.compile(model)

# Time eager
t0 = time.time()
for _ in range(50):
    model(x)
torch.cuda.synchronize()
print("Eager:", time.time() - t0)

# Time compiled
t0 = time.time()
for _ in range(50):
    compiled(x)
torch.cuda.synchronize()
print("Compiled:", time.time() - t0)
```

MIXED PRECISION TRAINING

Mixed precision training, as the name suggests, involves **mixing different precisions when training**. The idea of mixed-precision training is to use lower-precision formats for certain computations. **We can't fully abandon float32 and will usually need to perform some computations in full precision**.

Format	Total bits	Sign	Exponent	Mantissa
float32	32	1	8	23
float16	16	1	5	10
bfloat16	16	1	8	7
float8 (e4m3)	8	1	4	3
float8 (e5m2)	8	1	5	2

Mixed Precision Training | Explanation and PyTorch Implementation from Scratch, by **ExplainingAI**

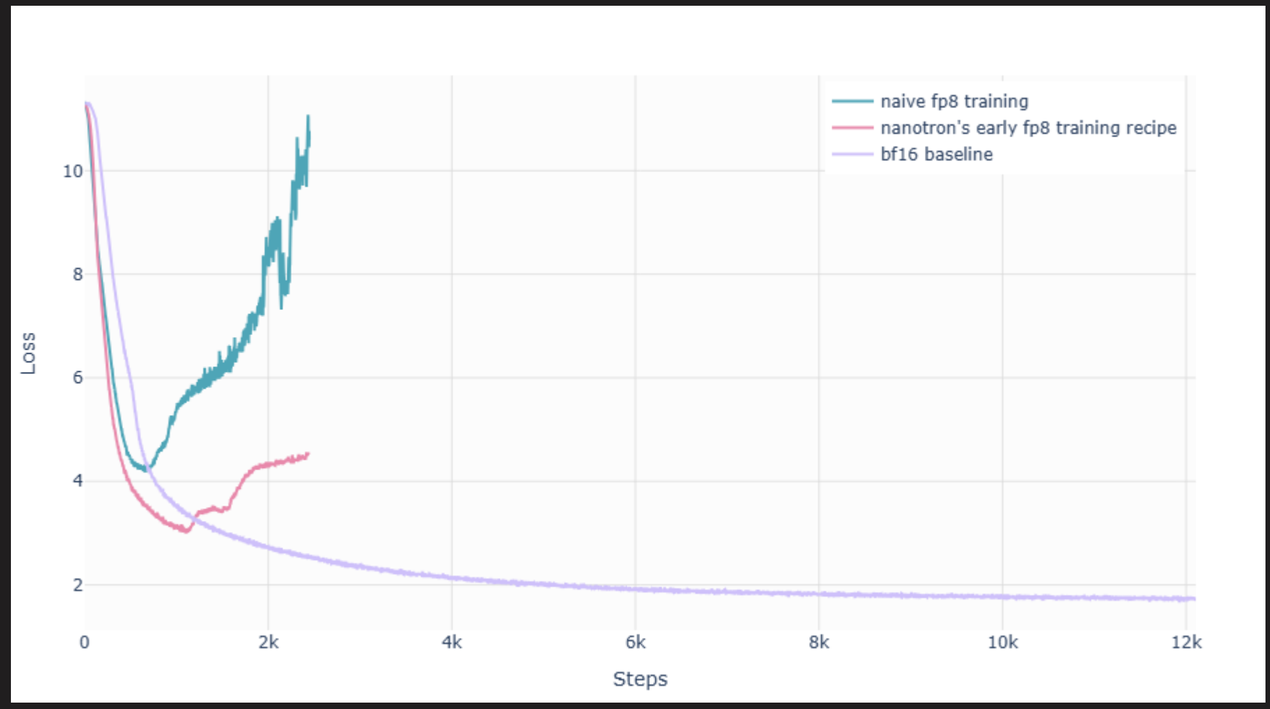
MIXED PRECISION TRAINING

FP16 and BF16 (the current standard):

- ✔ **FP32 master weights**
- ✔ **Loss scaling** (not required with bfloat16!)
- ✔ **FP32 accumulation** (for sums/averages)

FP8 Training (Experimental but very promising):

- ✔ H100: FP8 GEMMs ≈ 2× BF16 throughput
- ✔ Promising results from DeepSeek-V3.
- ✔ **Numerical stability is still a problem!**
- ✔ Likely future **replacement for BF16**



Source → The Ultra-Scale Playbook

MODEL CONFIGURATION AND OPTIMIZATION

- ✓ Learn how to initialize a distributed training environment using **PyTorch's DistributedDataParallel (DDP) with SLURM**.
- ✓ Learn how to initialize model architectures from scratch.
- ✓ Learn how to apply modern optimization techniques:
  - ✔ Liger Kernel optimizations (fused kernels for RoPE, RMSNorm, SwiGLU, cross-entropy)
  - ✔ Gradient checkpointing (activation recomputation) for memory efficiency
  - ✔ `torch.compile` for graph optimization

EXERCISE

BATCH SIZE

Efficiency and Criticality

SMALL KNOB, BIG CONSEQUENCES

The batch is one of the **most important hyperparameters** in training. It is a primary axis of parallelism and central to almost **every distributed setup** (DP is always in the mix).

Beyond wiring up distributed samplers and DDP (we'll get practical later), we need intuition for:

- ✔ *When does packing more samples into **one GPU** still make sense?*
- ✔ *When is it time to **add more GPUs** instead?*
- ✔ *When does a batch become **too big and start hurting performance**?*

FINDING AN EFFICIENT BATCH SIZE

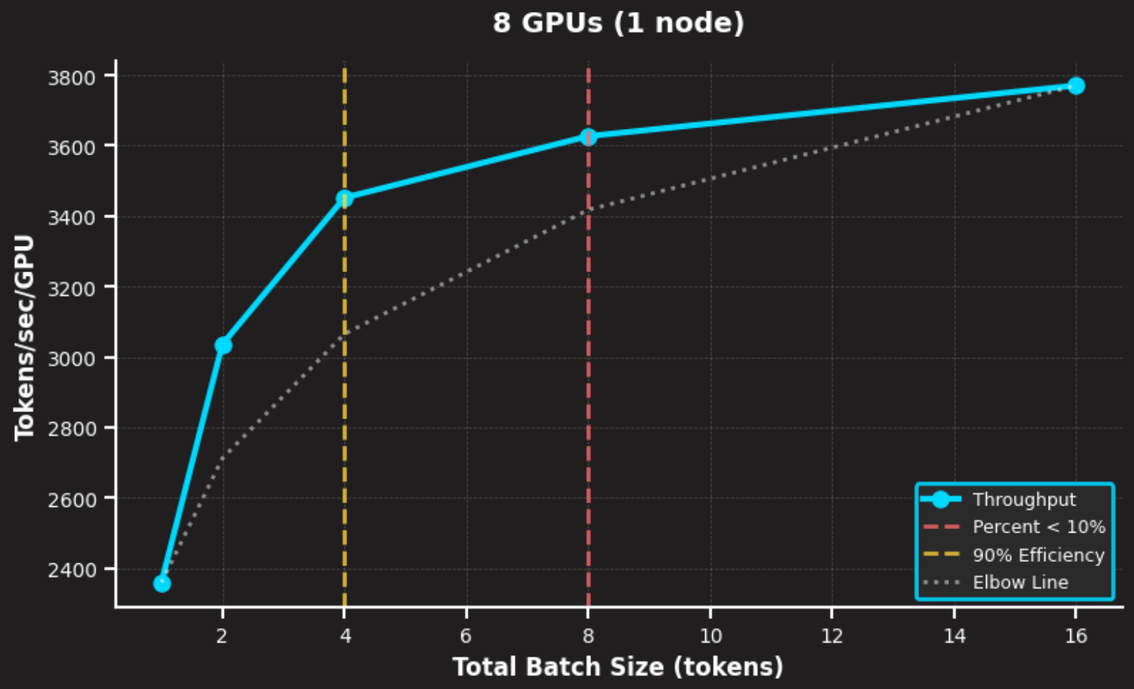
- ✓ A **small batch size can be useful** early in training to quickly traverse the training landscape and reach an optimal learning point (**warmup**). However, as training progresses, small batch sizes will **keep gradients noisy**, and the model may not converge to optimal performance. At the other extreme, **a large batch size**, while **yielding very accurate gradient estimates**, tends to require **fewer optimization steps**, which can also degrade convergence to a good minimum.

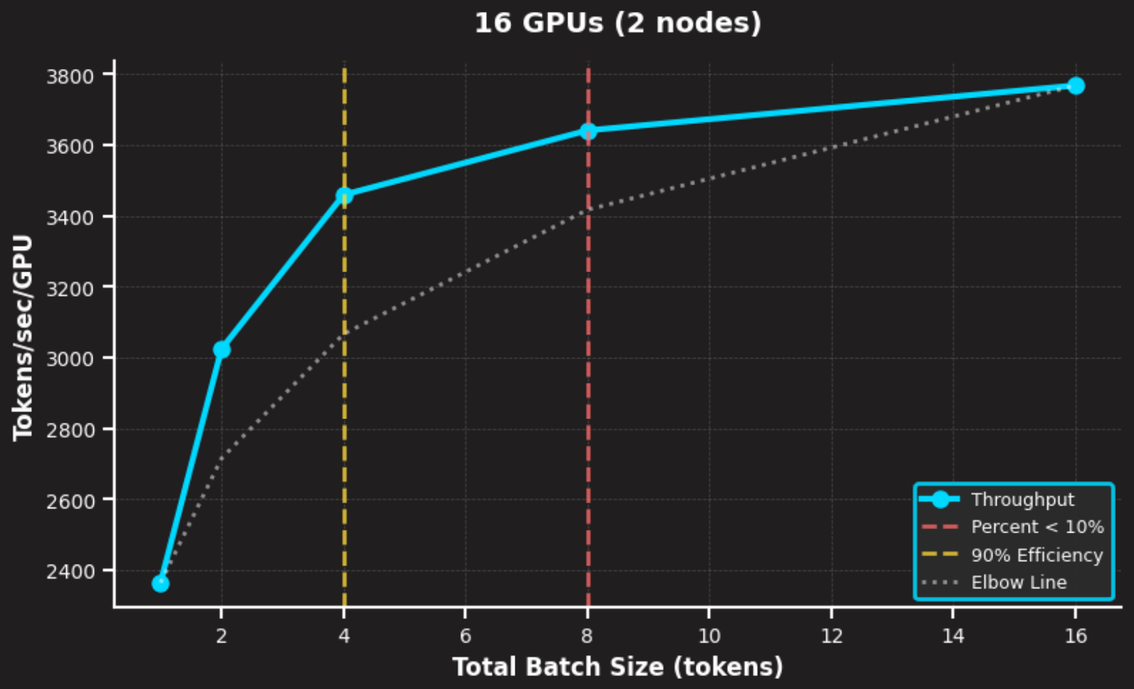
- ✓ That being said, the batch size **can often be adjusted quite widely around the optimal batch size without major impact on the performance of the model**, i.e., the sensitivity of final model performance to the exact batch size value is usually rather **low around the optimal batch size**.

💡 ***In LLM-land, batch sizes are reported in terms of tokens rather than the number of samples.***

FINDING AN EFFICIENT BATCH SIZE

When **scaling distributed training across multiple machines, maximizing your MFU** and **getting the job done** as quickly as possible are two different things when you have access to multiple hosts, and **we can scale DDP**.





FINDING AN EFFICIENT BATCH SIZE

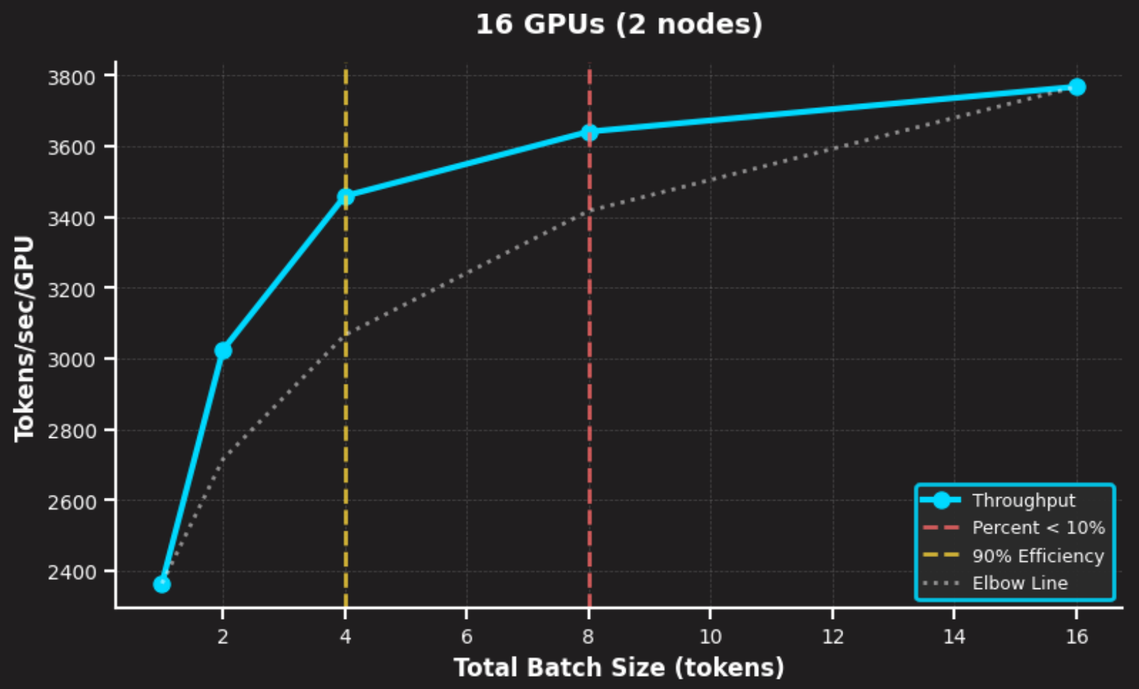
While scaling the DP dimension gives (almost!) **a linear increase** in performance (**AllReduce communication is fast**), naively increasing the micro batch **does not produce the same effect**.

Batch Size	Marginal Gain	Percent Gain (%)	Efficiency (%)	MFU (%)
1	–	–	0.62	34
2	677	28	0.80	44
3	208	13	0.91	50
4	43.75	5	0.96	53
5	18	4	1	55

*** FSDP / ZeRO Stage 2 / 7B Llama**

FINDING AN EFFICIENT BATCH SIZE

- ✓ **Percent Gain Threshold** → Find the point of diminishing returns by examining the percentage increase in throughput as batch size doubles (e.g., where this increase falls below a threshold 10%).
- ✓ **Efficiency Threshold** → Find the smallest batch size that achieves a target efficiency (e.g., 90%).
- ✓ **Elbow Method (Maximum Curvature)** → identifies the “knee point” in the throughput curve.



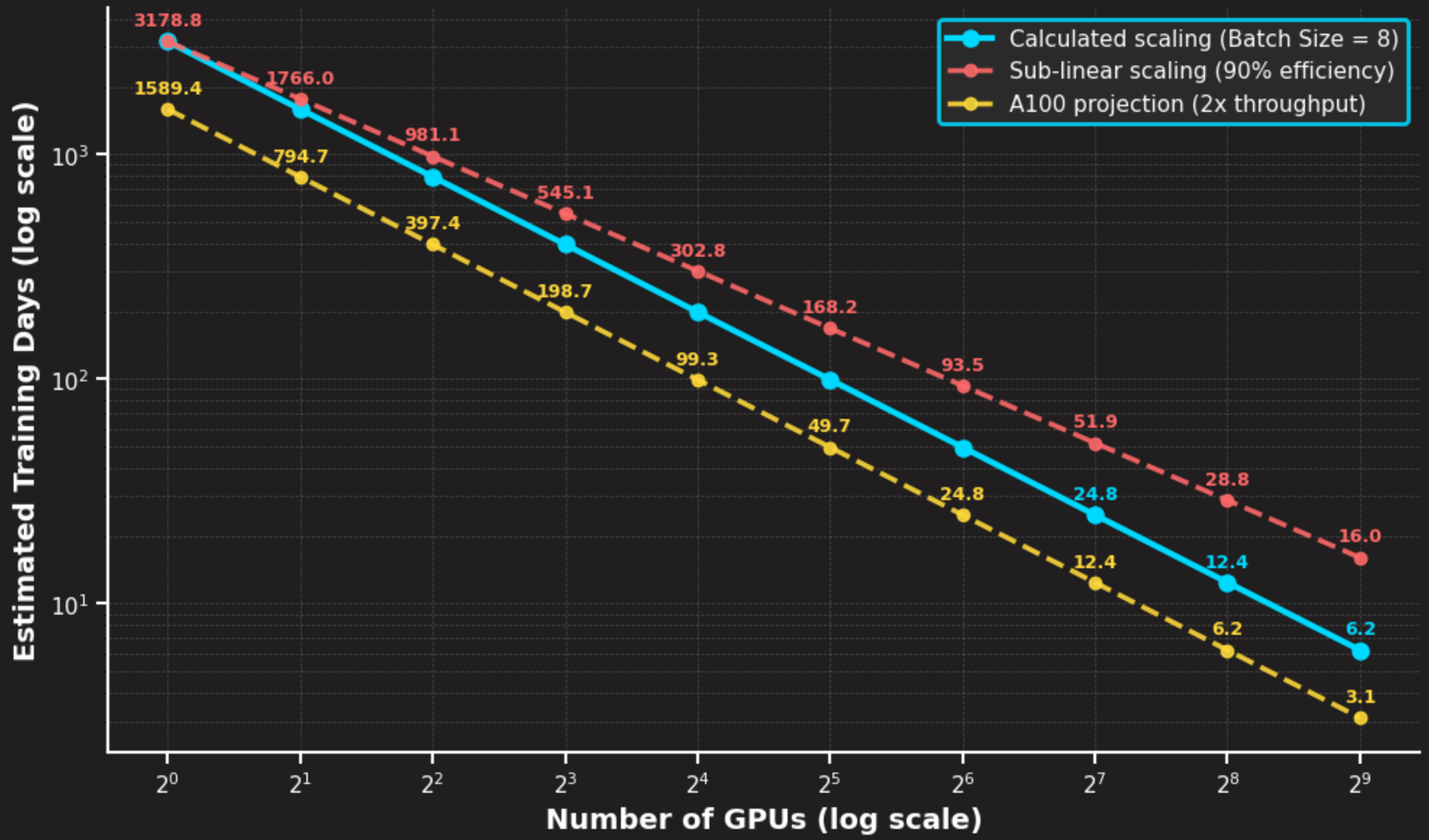
FINDING AN EFFICIENT BATCH SIZE

A quick recipe to find your efficient batch size:

1. **Collect throughput measurements** for a range of batch sizes (up to OOM).
2. You can choose one method, or use all three methods to triangulate an ideal batch size (**other methods exist!**).
3. Consider the training **context**:
   - ✔ For **hardware-constrained** scenarios, try to get the **most out of your GPU**.
   - ✔ If you are **not hardware-constrained**, **opt for "faster" batches at the cost of MFU**.

💡 Batch size efficiency **often remains consistent across similar GPU counts when working with simple Data Parallelism**, but as we add complexity, this may change, given that **communication patterns and overheads evolve**.

FINDING AN EFFICIENT BATCH SIZE



FINDING THE CRITICAL BATCH SIZE

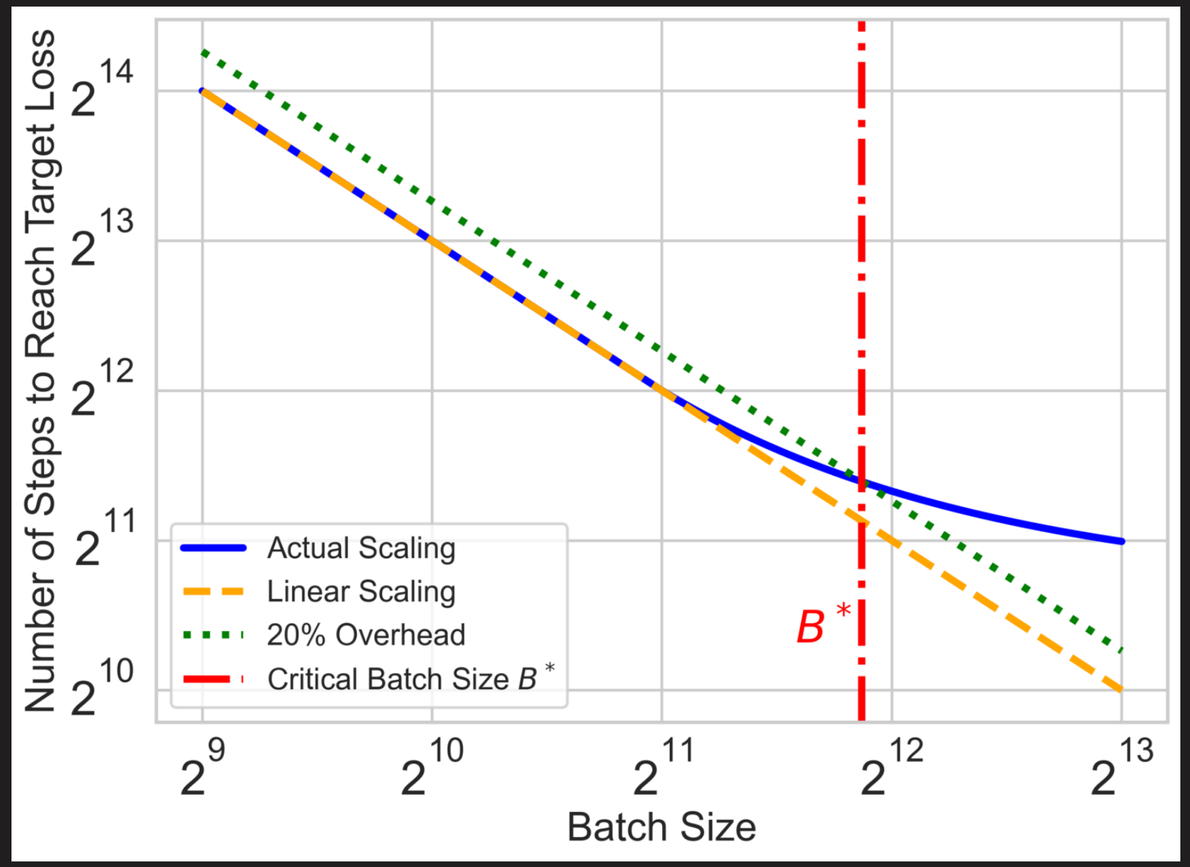
Batch size determines the number of samples processed before each weight update. Increasing it improves throughput as long as the hardware can keep up, but too-large batches reduce data efficiency—the model requires more total tokens to achieve the same loss. The dividing line is the **critical batch size (CBS)**.

- ✓ **Increasing the batch size while staying below critical:** after increasing the batch size and retuning the learning rate, you reach the same loss with the same number of tokens as the smaller batch size run, and no data is wasted.

- ✓ **Increasing the batch size while remaining above the critical value: larger batches sacrifice data efficiency; achieving** the same loss now requires more total tokens.

FINDING THE CRITICAL BATCH SIZE

- ✓ CBS is the batch size where the number of optimization steps to reach a target loss increases by 20% over the linear scaling regime (📎, 📎, 📎).
- ✓ Power-law recommendation by Zhang et al. (2024) — **derived from training 85 million to 1.2 billion parameters decoder-only transformers on the C4 corpora** — CBS ∝ D^0.47
- ✓ If you know the dataset size D (e.g., how many tokens), you can **predict CBS**.



How Does Critical Batch Size Scale in Pre-training? (Decoupling Data and Model Size), by Zhang et al.

FINDING THE CRITICAL BATCH SIZE

- ✓ If you double your dataset size **D,** your optimal batch size **B*** increases by about 2^0.47 (~1.38 times bigger), **not double**. This sub-linear growth means the batch size **grows more slowly than the dataset size**.

- ✓ For example, keeping model (**N**) constant, if our dataset has 500B tokens and we already know that, with a batch size of 2M, we are under CBS, we can predict the new CBS with the larger dataset size as:

  ✔ `2e+6 * 1.38 ~ 2.76e+6.`

💡 **Rule of thumb: CBS scales with data, not model size**. When both model size (**N**) and data size (**D**) increase proportionally, CBS increases. If you fix the model size and increase data size, CBS rises. But if you fix data size and scale model size, CBS remains roughly constant.

NOTE: CRITICAL BATCH SIZE CAN GROW

Some large-scale training therefore uses a **batch-size warmup**, increasing the batch size as training stabilizes (e.g., DeepSeek-V3 grows from **12.6M** to **62.9M** tokens per batch). **Keeping LR constant, batch increases mimic LR decay.**

- ✔ Early training: gradient norms are large → **critical batch size is small**.
- ✔ Late training: gradients shrink → **larger batches become effective**.

GRADIENT ACCUMULATION

Gradient Accumulation = Infinite Batch Size*

- ✔ Split one big batch into tiny **micro-batches**
- ✔ Run forward + backward on each
- ✔ Average the gradients → **update once**
- ✔ This is **sequential** by necessity.
- ✔ Same result*. **Less memory**.
- ✔ Works great with **recomputation**.

How It's Used in Practice

- ✔ **First: scale with data parallelism** (parallel)
- ✔ **Then: add gradient accumulation** (sequential)
- ✔ **Together → reach your target global batch size**

DATASETS AND DISTRIBUTED DATA LOADERS

- ✔ Learn how to create distributed samplers to partition datasets across multiple GPUs.
- ✔ Learn how to implement custom collate functions for language modeling.
- ✔ Learn how to configure `DataLoaders` with proper settings (prefetching, pinned memory, workers).

EXERCISE

TRAINING STABILITY

Optimizers, Schedulers, and other tricks

AGENDA

4. Training Stability

- ✓ Measuring Training Stability
- ✓ Keeping Things Stable
- ✓ Optimizers
- ✓ Learning Rate Scheduling
- ✓ The DeepSeek Heuristic

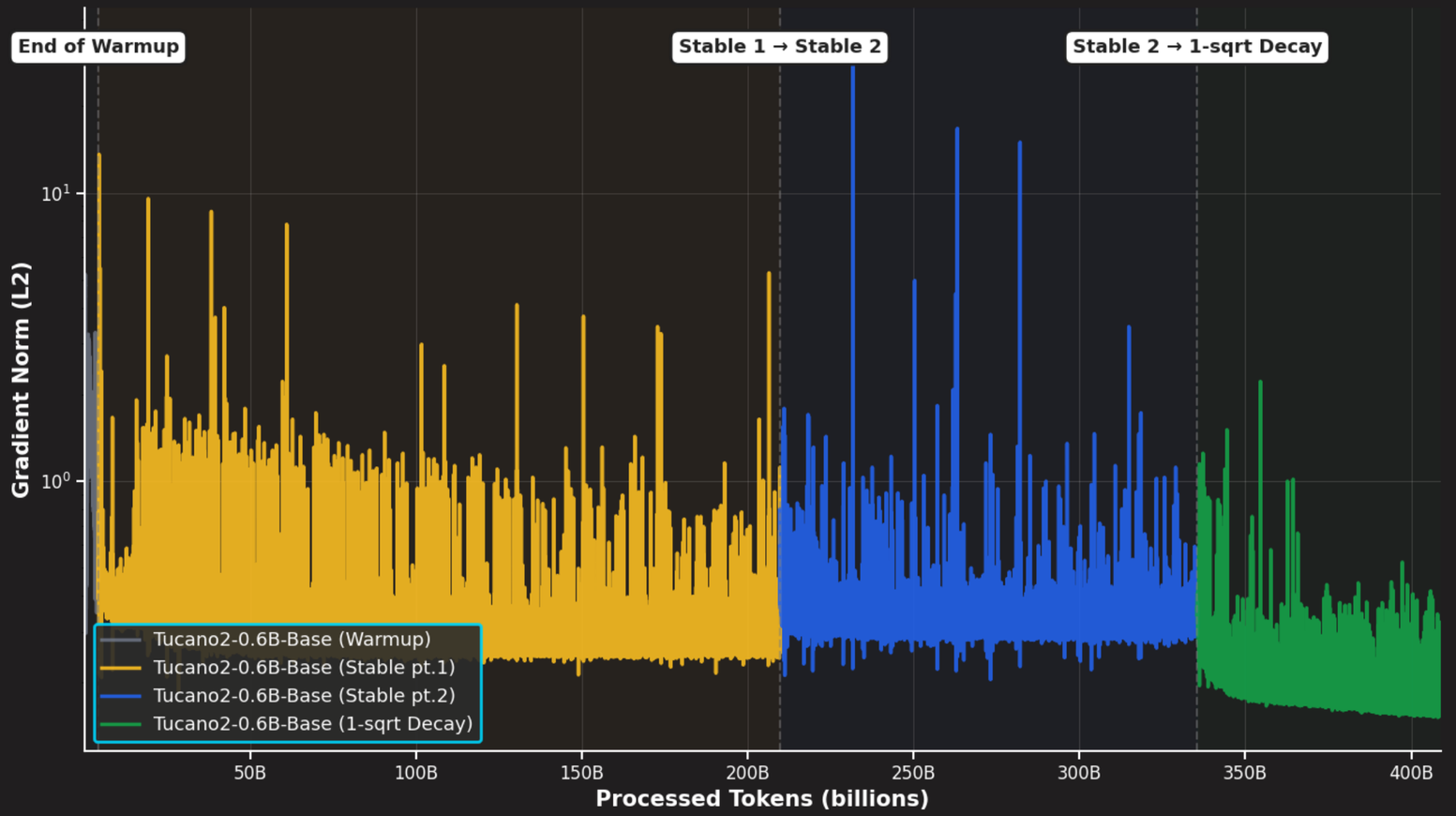
TRAINING STABILITY

- ✓ There is a reason we often call pre-training runs *marathons*. Unlike many deep learning workloads (such as training a digit classifier on MNIST), pretraining involves **extremely long training runs**. Long runs entail many optimization steps, and more steps increase the likelihood of issues. As a result, numerical issues and training instabilities become a central concern.

- ✓ The Transformer architecture is, by design, a **relatively stable neural network**. Skip connections and the pervasive use of normalization techniques help keep activations within healthy ranges and allow gradients to flow smoothly. Nevertheless, **instabilities can and do occur, and managing them is critical for successful pretraining**—especially as model size and training scale increase.

MEASURING TRAINING STABILITY

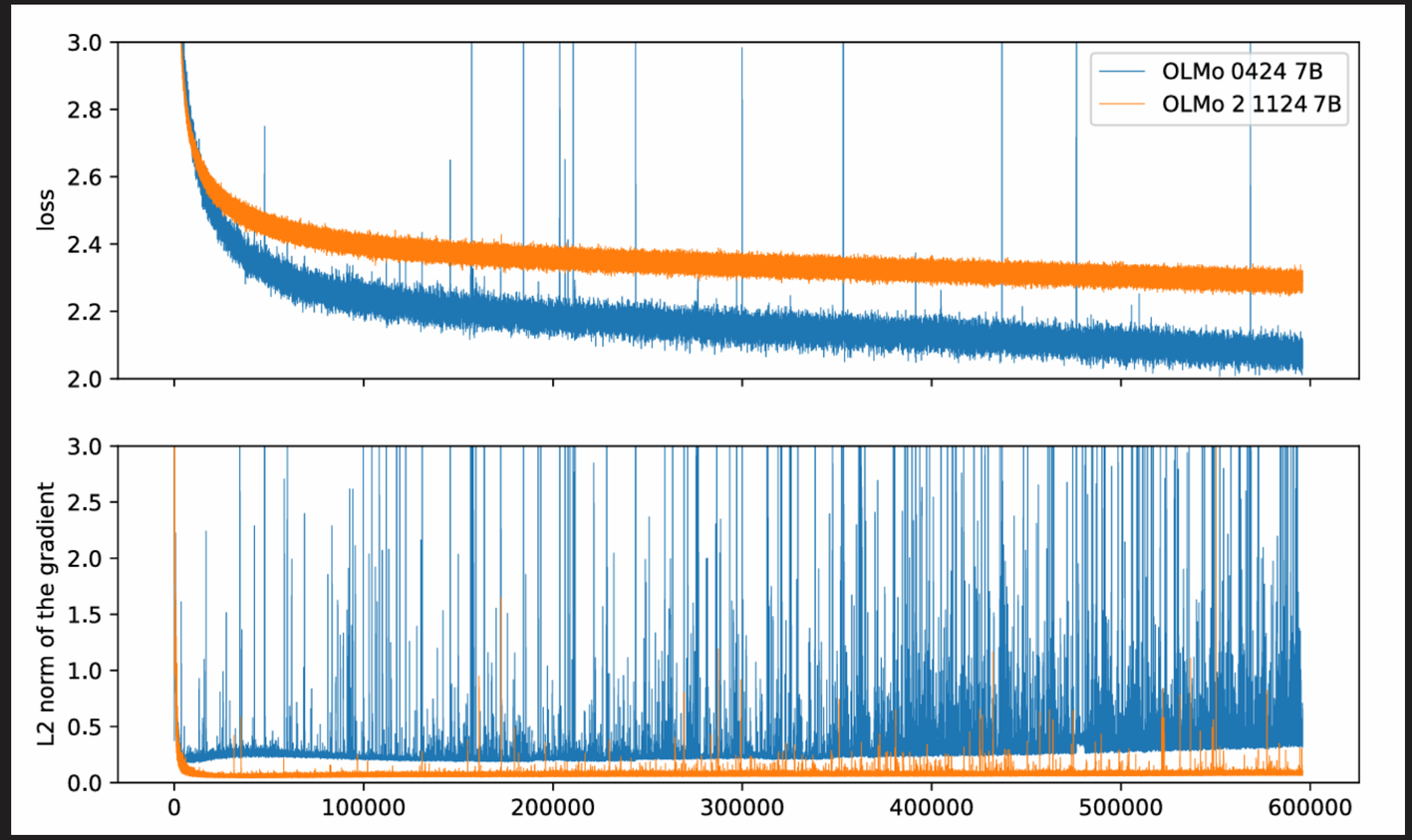
- ✓ The **L2 norm** (also called the *Euclidean norm*) is a way to measure the **length or magnitude of a vector**.
- ✓ Flatten gradients into a 1D vector → Square each component → Summ → Take a square root → **L2 Norm!**
- ✓ ***How strongly are gradients being updated?***



MEASURING TRAINING STABILITY

- ✓ Keeping the **GradNorm** in check is one of the main things we need to be focusing on, especially during early **ablations**, **where we are testing hyperparameters**.

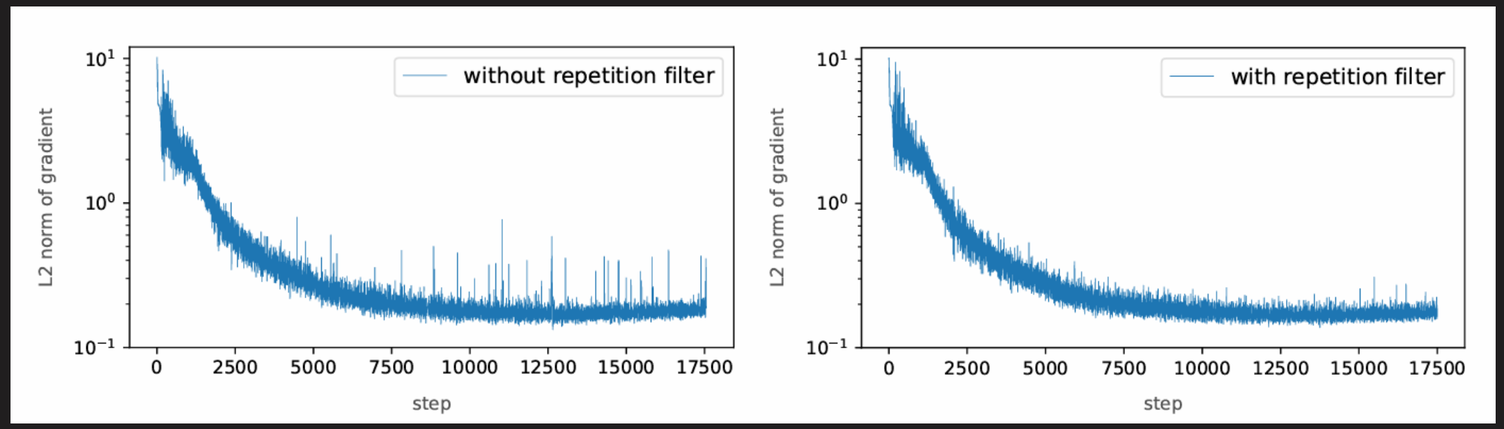
- ✓ Optimizing for a more stable training has been shown to **directly correlate with final model performance**.



Source → 2 OLMo 2 Furious

KEEPING THINGS STABLE

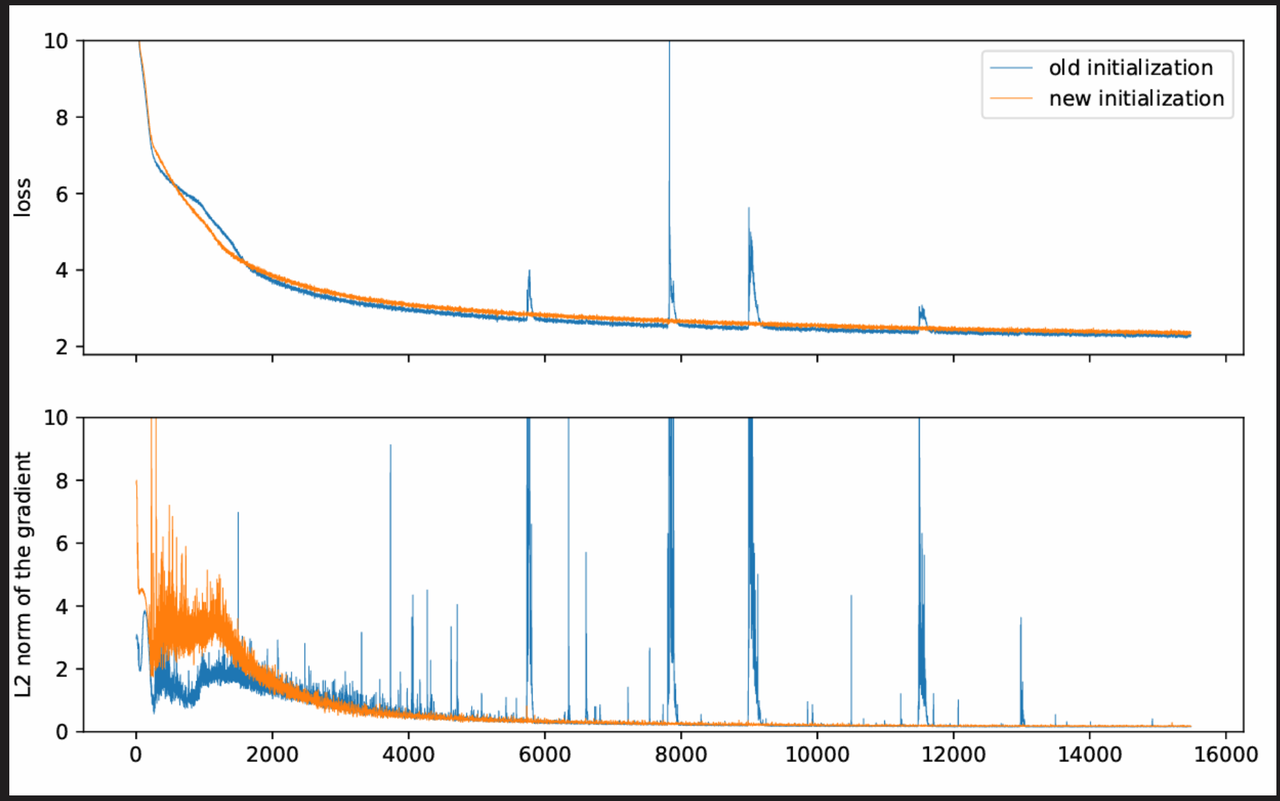
- ✓ Samples with a load of repeated grams (g4ODg4ODg4ODg4ODg4ODg4ODg4ODg4ODg4ODg4ODg4ODg4ODg4ODg4OD) can cause **loss spikes**.
- ✓ If you are using a dataset you did not create/filtered, **make sure to perform the proper quality checks**.



Source → 2 OLMo 2 Furious

KEEPING THINGS STABLE

- ✓ The range in which parameters are initialized has a very strong effect on training stability.
- ✓ Scaled initialization (Zhang et al., 2019) VS. mean of 0 and a standard deviation of 0.02.
- ✓ Most documented training runs report using an initializer range of 0.02 nowadays.



Source → 2 OLMo 2 Furious

KEEPING THINGS STABLE

✓ **Where is the optimal placement of LayerNorms???**

```
# Gemma3 (Gemma3DecoderLayer)
...
hidden_states = self.post_attention_layernorm(hidden_states)
hidden_states = residual + hidden_states

residual = hidden_states
hidden_states = self.pre_feedforward_layernorm(hidden_states)
hidden_states = self.mlp(hidden_states)
hidden_states = self.post_feedforward_layernorm(hidden_states)
hidden_states = residual + hidden_states

return hidden_states
```

Gemma3 (🔗), Qwen3 (🔗), Olmo2 (🔗)

KEEPING THINGS STABLE

✓ **Where is the optimal placement of LayerNorms???**

```
# Olmo2 (Olmo2DecoderLayer)
...
hidden_states = self.post_attention_layernorm(hidden_states)
hidden_states = residual + hidden_states

residual = hidden_states
hidden_states = self.mlp(hidden_states)
hidden_states = self.post_feedforward_layernorm(hidden_states)
hidden_states = residual + hidden_states
return hidden_states
```

Gemma3 (🔗), Qwen3 (🔗), Olmo2 (🔗)

KEEPING THINGS STABLE

✓ **Where is the optimal placement of LayerNorms???**

```
# Qwen3 (Qwen3DecoderLayer)
...
hidden_states = residual + hidden_states

residual = hidden_states
hidden_states = self.post_attention_layernorm(hidden_states)
hidden_states = self.mlp(hidden_states)
hidden_states = residual + hidden_states
return hidden_states
```

Gemma3 (🔗), Qwen3 (🔗), Olmo2 (🔗), and QK-Norm are also a thing!

OPTIMIZERS

Before we can keep talking about tricks and hacks to keep your training stable, we need to talk about **optimizers**. The optimizer sits at the **heart of the entire LLM training pipeline**. It determines, for each parameter, the update step by combining information from past updates, the current parameter values, and gradients computed from the loss.

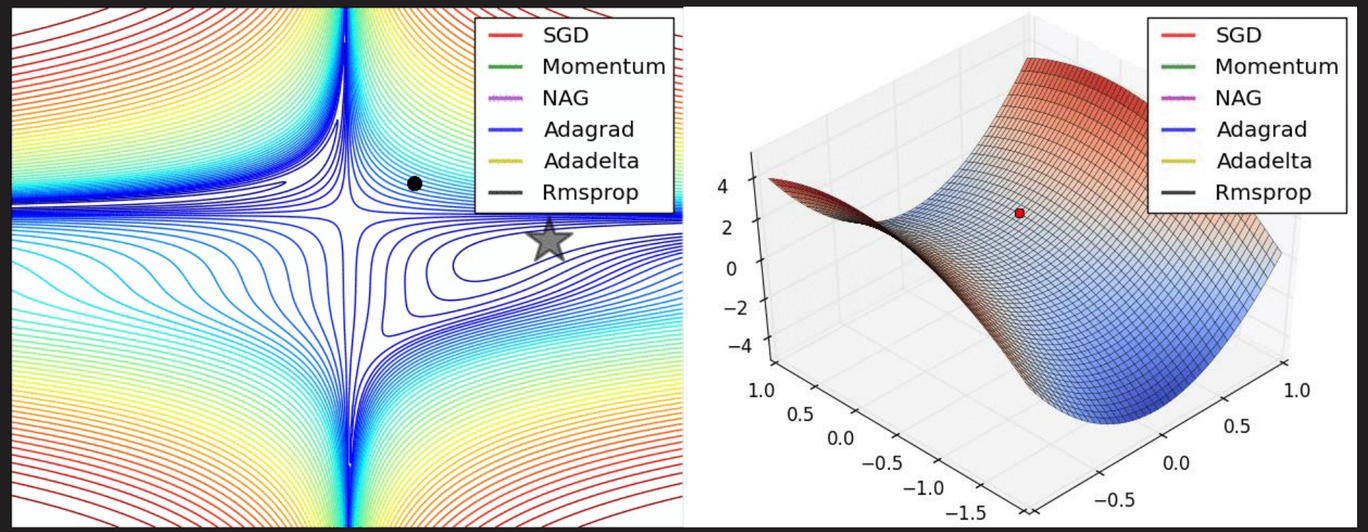
- ✔ Optimizers (although some more than others) are memory- and compute-hungry beasts.
- ✔ Must read: **An overview of gradient descent optimization algorithms**, by Sebastian Ruder.

Model	Optimizer
Kimi K2, GLM	Muon
Everyone else	AdamW

OPTIMIZERS

Should I ablate optimizers? If you are not up for some hard work, **no**.

- ✓ Comparing optimizers **fairly is hard**.
- ✓ **Scale** changes everything.
- ✓ Each optimizer **requires tuning**.
- ✓ Optimizer research is **costly**.



Source → An overview of gradient descent optimization algorithms (taken from Alec Radford)

OPTIMIZERS / ADAM(W)

- ✓ **Adaptive Momentum Estimation** is a first-order optimization technique. In addition to examining the gradients alone, we also consider how much the weights changed in previous steps. This makes the learning rate for each parameter adapt based on the momentum.

- ✓ **Weight decay** is a regularization technique that penalizes large weights by slightly shrinking them during training. Improve **generalization and reduce overfitting**.

```
# theta: current weights
# lr: learning rate
# grad: gradient of the loss w.r.t weights
# wd: weight decay coefficient

# Standard Adam update without weight decay
theta = theta - (lr * grad)

# With weight decay (decoupled, as in AdamW)
theta = theta - (lr * grad) - (lr * wd * theta)
```

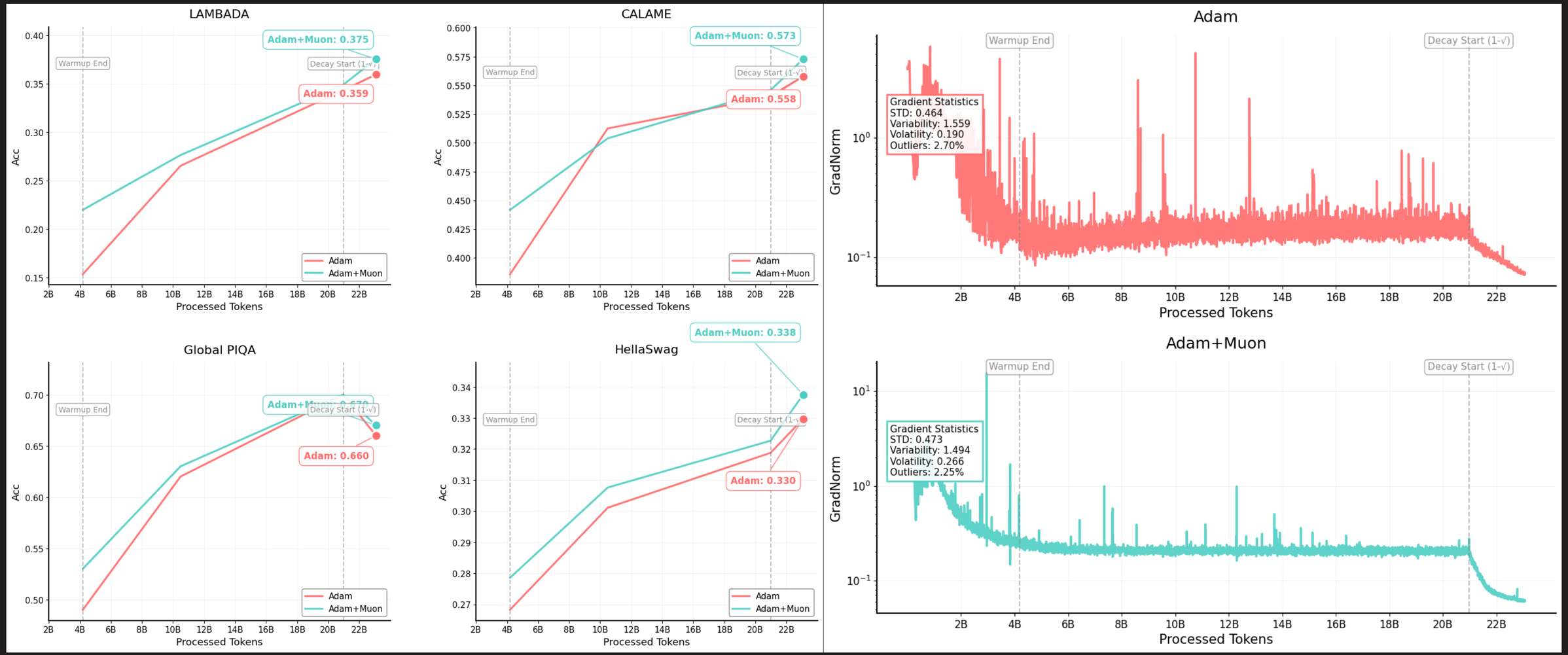
OPTIMIZERS / MUON

- ✓ **Muon** is a *second-order* optimizer that operates on the **matrix structure** of each weight tensor, not individual parameters.
- ✓ Uses the **Newton–Schulz iteration** of order 5 to approximate the matrix inverse square root (fast!).
- ✓ Newton–Schulz yields an approximation of the **SVD** of a **2D matrix**.
- ✓ It **promotes exploration** of directions suppressed by tiny singular values.
- ✓ It is **highly stable** (even when batch sizes are large).

```python
def zeropower_via_newtonschulz5(G, steps=5):
    """
    Newton-Schulz iteration to compute the zeroth power / orthogonalization of G.
    """
    assert G.ndim >= 2
    a, b, c = (3.4445, -4.7750,  2.0315)
    X = G.bfloat16()
    if G.size(-2) > G.size(-1):
        X = X.mT

    # Ensure spectral norm is at most 1
    X = X / (X.norm(dim=(-2, -1), keepdim=True) + 1e-7)
    # Perform the NS iterations
    for _ in range(steps):
        A = X @ X.mT
        B = b * A + c * A @ A
        X = a * X + B @ X

    if G.size(-2) > G.size(-1):
        X = X.mT
    return X
```

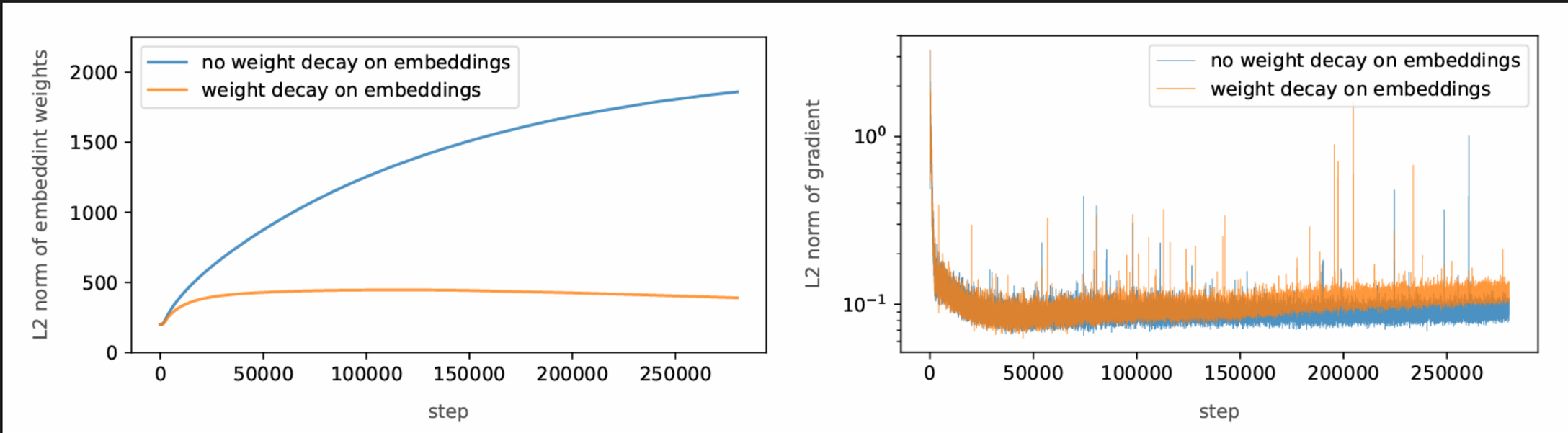
Source → Muon: An optimizer for hidden layers in neural networks, by Keller Jordan

MUON IS VERY PROMISING ...



KEEPING THINGS STABLE

- ✓ **Excluding embeddings from weight decay** can improve training stability. The reasoning is that weight decay causes **embedding norms to gradually decrease** during training.
- ✓ This means that the weight decay slowly **erases differences in embeddings** that should be preserved, which causes larger gradient norms at the end of training, because the model is **struggling to differentiate them/make sense of them**.



Source → 2 OLMo 2 Furious

KEEPING THINGS STABLE

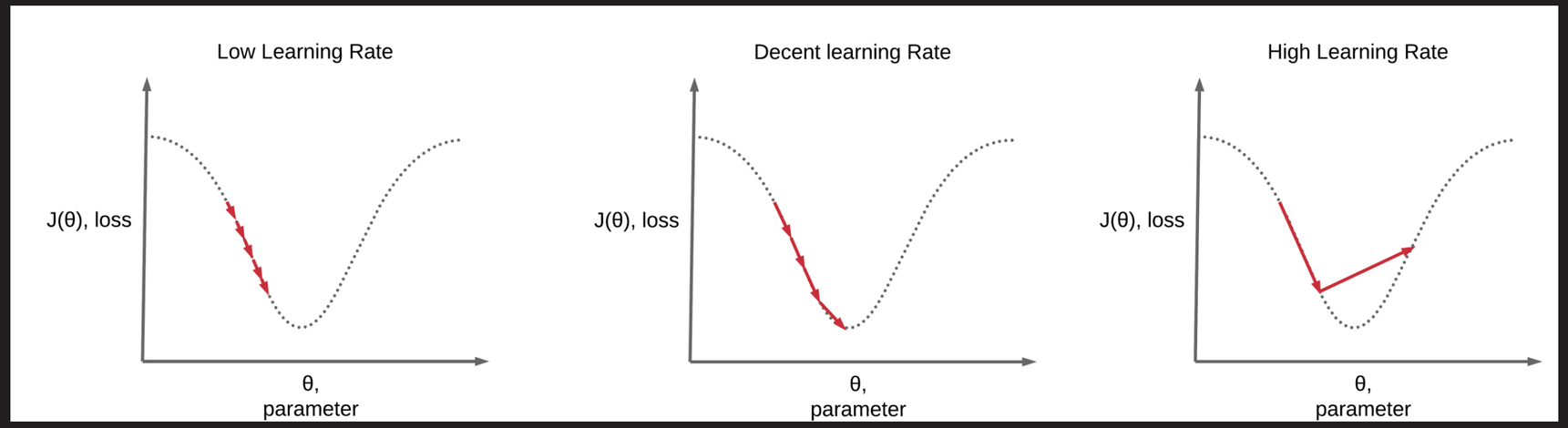
- ✓ **Just force the gradients to be small!**
- ✓ Gradient clipping is a technique used to prevent exploding gradients during training. By **capping the maximum allowable gradient norm**, we can ensure that model parameter updates remain within a reasonable range.
- ✓ As model size increases, you can **increase the max grad norm**.
- ✓ If things start to explode, **lower the max grad norm**.

```
for batch in data_loader:
    optimizer.zero_grad()
    outputs = model(batch)
    loss = loss_fn(outputs, targets)
    loss.backward()

    # Gradient clipping
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    # What is this doing?
    # Get the total norm, Get the max norm, Get your scale
    # scale = min(1, max_norm / (total_norm + 1e-6))
    # clipped_grad = grad * scale
    # All gradients are scaled together to preserve the direction!

    optimizer.step()
```

Source → torch.nn.utils.clip_grad_norm

LEARNING RATE SCHEDULING

The learning rate is one of the **superstar hyperparameters**. Think of it as the volume knob on how aggressively our model updates its weights with each training step. Turn it too low, and **training drags along at a snail's pace**, and we'll chew through our compute budget without actually making meaningful progress. Crank it too high, though, and our **optimizer starts taking wild leaps, and the loss skyrockets to infinity**. Finding the right **balance is key**.

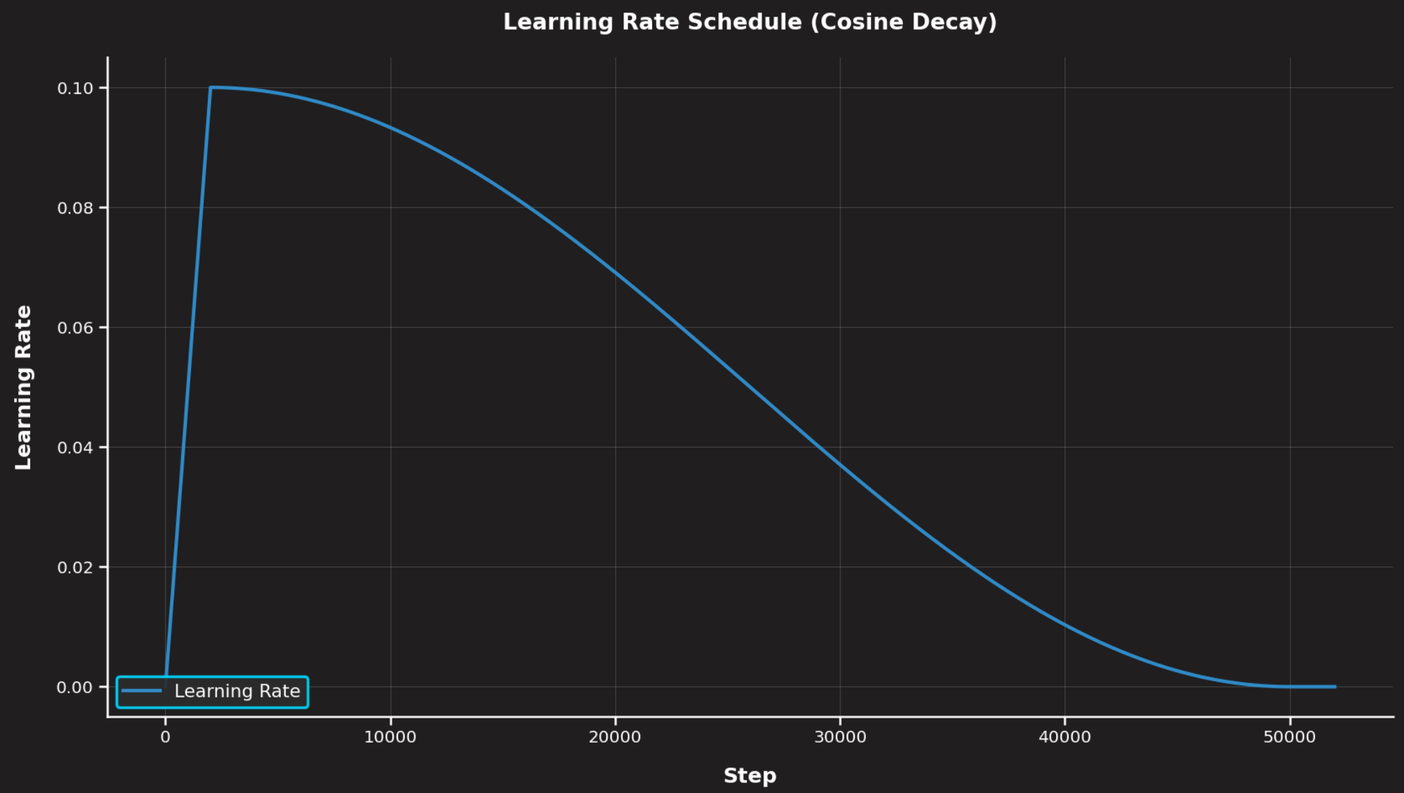


Source → Deep Learning Wizard: Learning Rate Scheduling

VANILLA WARM UP + COSINE DECAY

- ✓ Warmup from zero to **avoid early chaos**, then decay to settle into a good minimum. These patterns (warmup + something) have been **validated for neural network training for many years (,).**

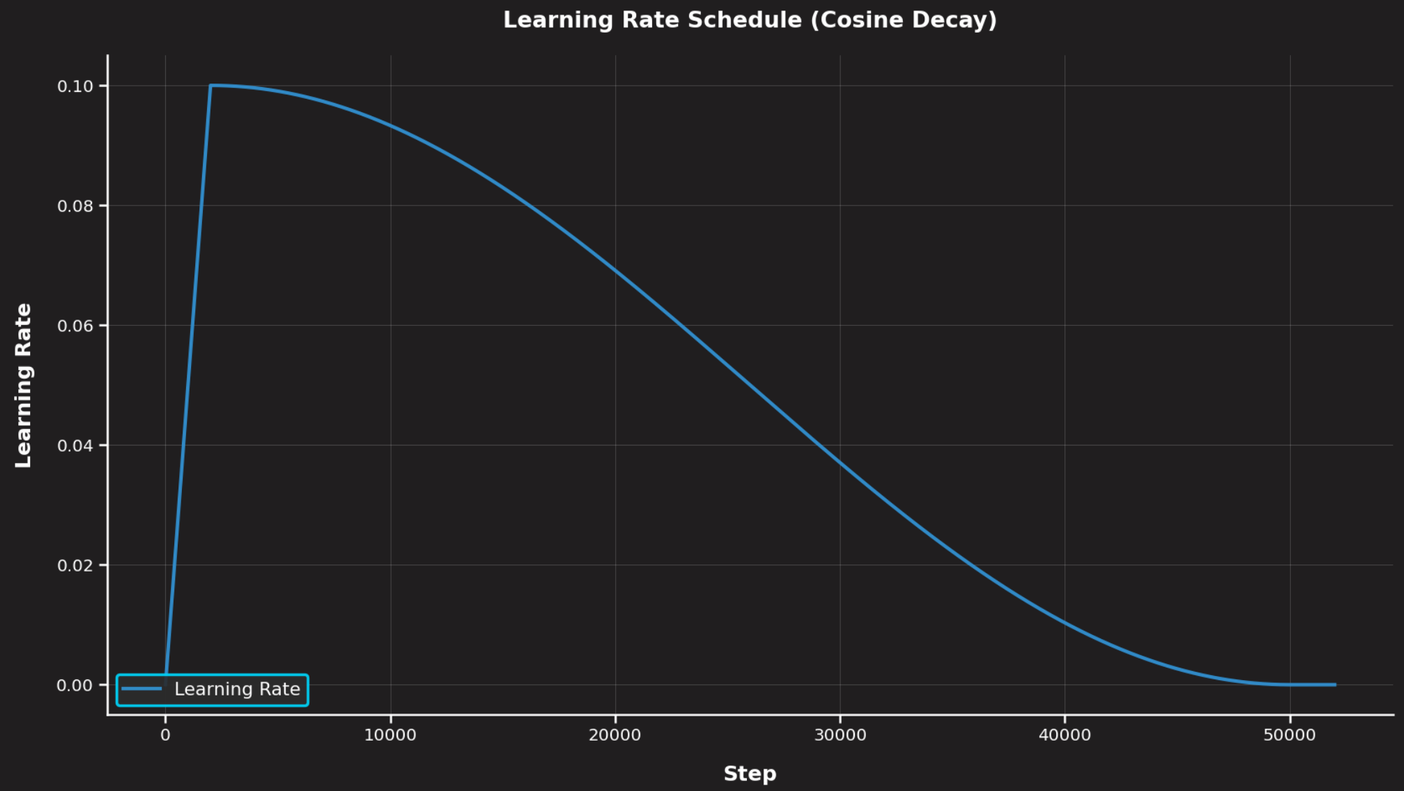
- ✓ Across the literature (Llama, OPT, GT3, DeepSeek, Smollm, etc.), you will find **the recurrence of the following values**:
  - ✔ Warmup-steps: **1000 – 2000 steps**
  - ✔ Cosine decay: **80% - 100%**
  - ✔ Stable: **10% – 20%**



VANILLA WARM UP + COSINE DECAY

- ✓ We need to **know our total training steps upfront**, as the cosine cycle length must match your total training duration. This becomes a problem when you want to change settings during training (e.g., **you get more compute and want to train longer**).

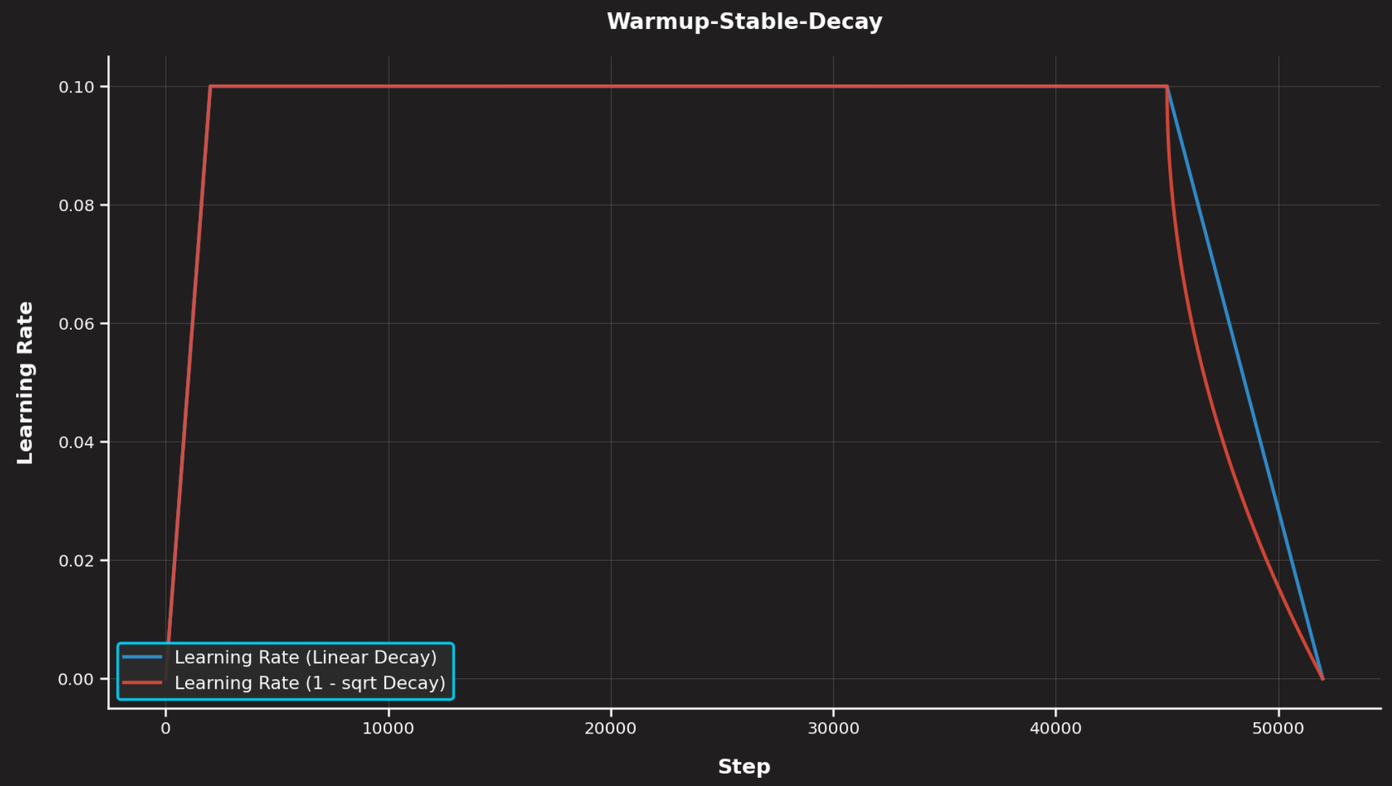
- ✓ Cosine decay **forces you to restart from scratch.**



WARMUP-STABLE-DECAY

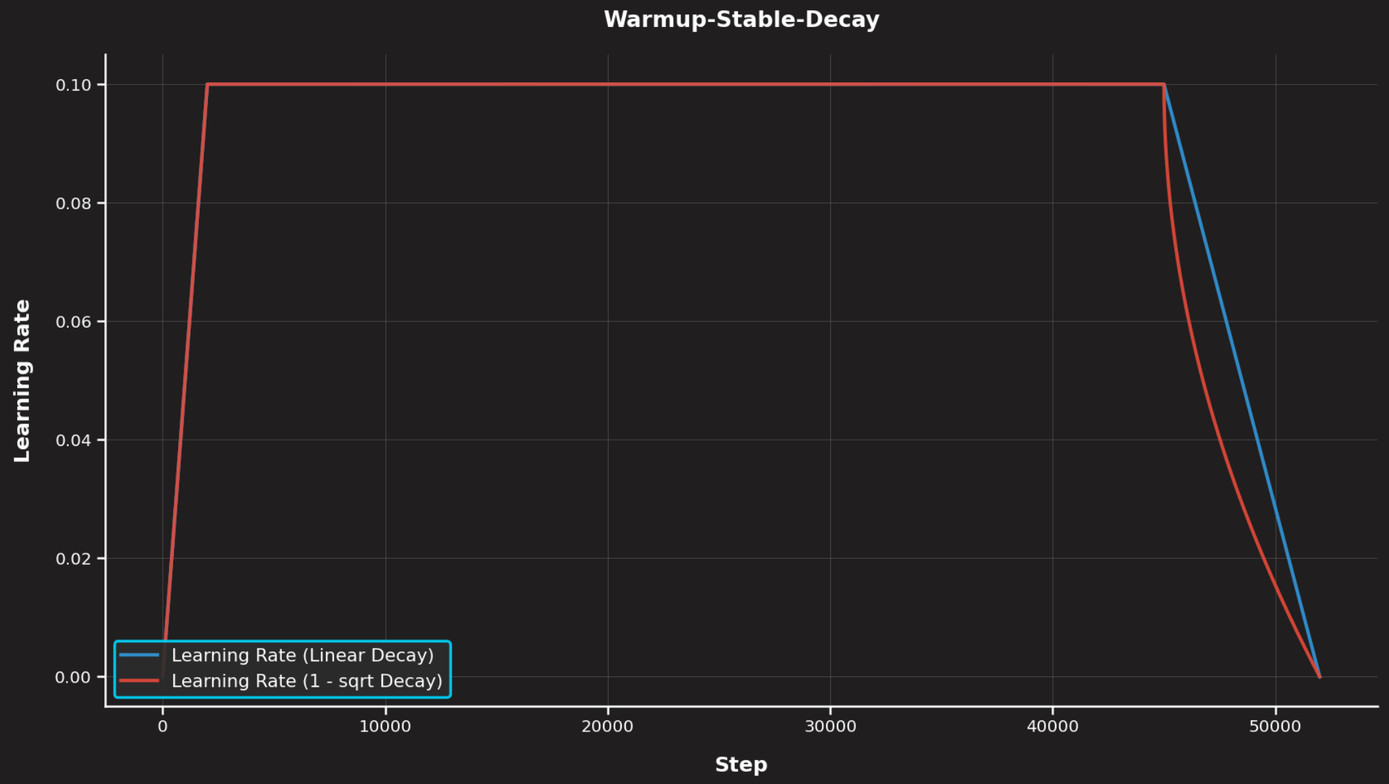
- ✓ Many teams now use schedules where you don’t need to start decaying immediately after warmup. This is the case for **Warmup-Stable-Decay (** WSD) (🧶) and **Multi-Step** (🧶).

- ✓ These schedules offer practical advantages over cosine decay. **We can extend training mid-run without restarting**, whether we want to train longer than initially planned or **decay early to measure training progress**.



WARMUP-STABLE-DECAY

- ✓ The required cooldown duration to match cosine performance decreases with longer training runs, and it is **recommended to allocate 10-20% of total tokens to the decay phase** (🧶).
- ✓ For Multi-Step, ablations found that proportions like 70/15/15 and 60/20/20 splits can **outperform cousine** (🧶).
- ✓ **Several studies confirm that 1-sqrt decay is superior to linear decay (🧶, 🧶, 🧶).**



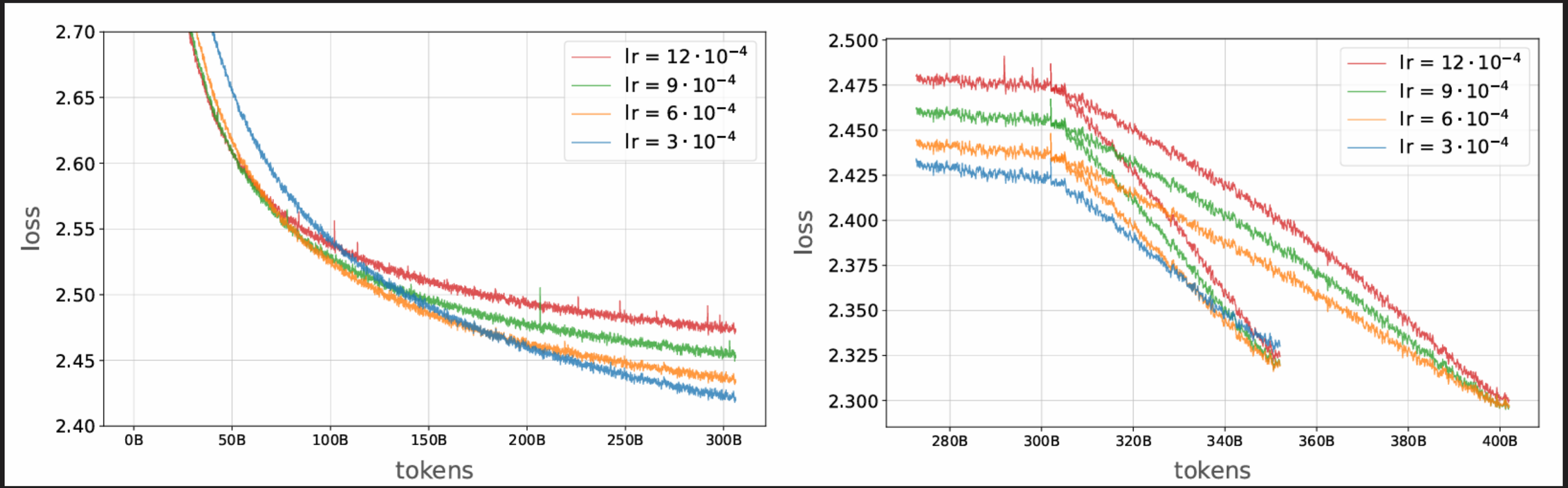
THE DEEPSEEK HEURISTIC FOR BS AND LR

- ✓ How do we **choose the right learning rate for a given scheduler and training setup**?

- ✓ One approach is to **run learning rate sweeps on short ablation experiments**. However, the optimal learning rate often depends on the total training duration: **a learning rate that appears to converge fastest in a short trial might not perform best over the full run**. And running multiple expensive, **multi-week trainings just to test different learning rates is usually out of the question**.

- ✓ Sweeps are useful to rule out learning rates that are clearly too high (**divergence**) or too low (**slow convergence**).

THE DEEPSEEK HEURISTIC FOR BS AND LR



Source → 2 OLMo 2 Furious

THE DEEPSEEK HEURISTIC FOR BS AND LR

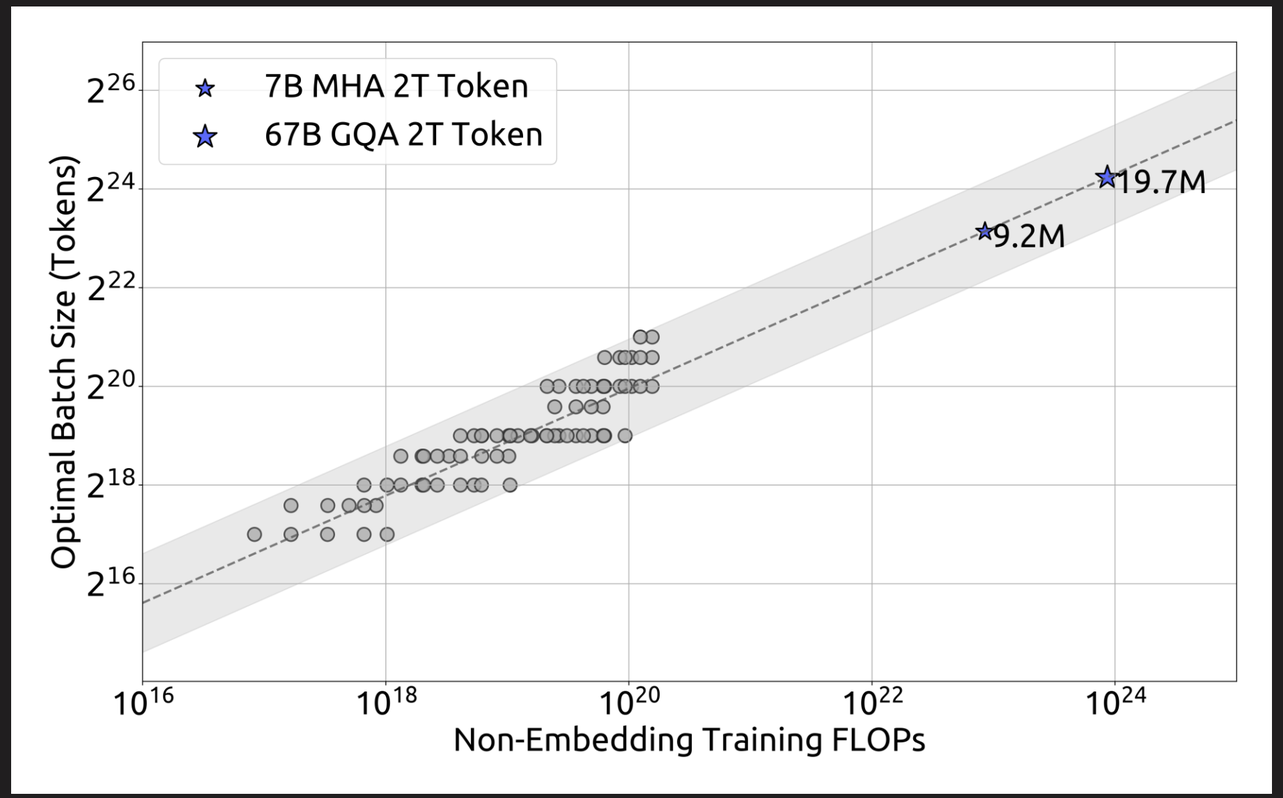
- ✓ Scaling laws **establish empirical relationships describing how model performance evolves as we increase training scale**, whether that's through larger models or more training data.

- ✓ But scaling laws can also help us **predict how to adjust key hyperparameters** like the learning rate and batch size as we scale up training (📎, 📎).

- ✓ This gives us principled defaults **rather than relying entirely on hyperparameter sweeps (📎, 📎, 📎)**.

THE DEEPSEEK HEURISTIC FOR BS AND LR

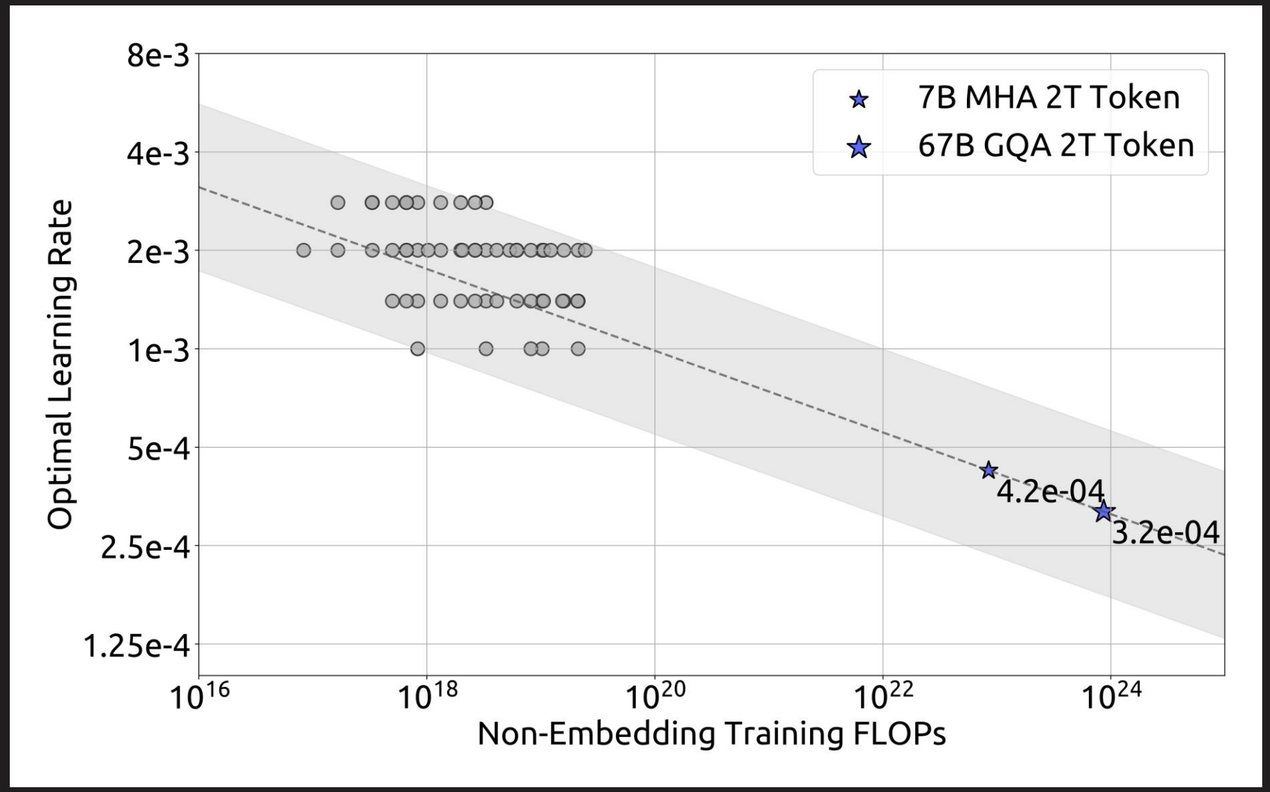
- ✓ *"Through extensive experiments, we modeled the power law relationship between the compute budget CC and the optimal batch size and learning rate. This relationship, which we refer to as the scaling laws of hyperparameters, provides an empirical framework for determining the optimal hyperparameters."* (DeepSeek-AI, 2024)



Source → DeepSeek LLM: Scaling Open-Source Language Models with Longtermism

THE DEEPSEEK HEURISTIC FOR BS AND LR

- ✓ On a **log-log scale**, optimal learning rate and batch size follow **power-law trends**. There is a **broad sweet spot** for hyperparameters, i.e., performance is stable across a wide range, so exact tuning is less critical.
- ✓ The formulas predict **smaller learning rates** for larger training runs (stability at scale).
- ✓ The **batch size grows** with compute (taking advantage of parallelism).



Source → DeepSeek LLM: Scaling Open-Source Language Models with Longtermism

THE DEEPSEEK HEURISTIC FOR BS AND LR

```
N = 670_000_000          # Model parameters (≈670M)
D = 230_000_000_000      # Training tokens (230B)

hidden_size = 1536
num_hidden_layers = 28
max_position_embeddings = 4096

# Flops per token (M) based on transformer architecture
M = 72 * num_hidden_layers * (hidden_size ** 2) \
    + 12 * num_hidden_layers * hidden_size * max_position_embeddings

# The M-based C nudges LR down and BS up.
# It is more conservative and typically more
# representative of training runs done with
# longer contexts.
C = M * D
lr = 0.3118 * (C ** (-0.1250))
bs = 0.2920 * (C ** (0.3271))

print(lr, bs)
#>> 0.00069829415380704 62 2508869.9932269943
```

OPTIMIZER AND LEARNING RATE SCHEDULER

- ✔ Learn how to implement learning rate schedulers from.
- ✔ Learn how to create parameter groups for differential weight decay.
- ✔ Learn how to configure the AdamW optimizer with proper hyperparameters.
- ✔ Learn how to compile the optimizer step function for additional performance gains.
- ✔ Learn how to set up MFU calculation constants for monitoring hardware efficiency.

EXERCISE

UNFORTUNATELY, BEYOND THE SCOPE

But That Does Not Mean You Should Not Know About

CONTINUAL PRETRAINING

- ✓ In continual pretraining, we **typically initialize from a model checkpoint that has already been trained on a large corpus**. Because the model has learned useful representations, we must be careful not to **perturb these weights too aggressively**.

- ✓ Continual pretraining from base models whose original training conditions are **largely unknown**—such as the learning rate at the end of training—**introduces an additional layer of difficulty**. Many established heuristics and scaling laws must therefore be adapted specifically for the continual pretraining (CPT) setting.

CONTINUAL PRETRAINING

Reference	What they say / why it's relevant
Reuse, Don't Retrain: A Recipe for Continued Pretraining of Language Models (Parmar et al. 2024)	A foundational "recipe" for CPT: they show that carefully choosing the learning-rate schedule (while re-using a pretrained model) **yields substantial gains vs naive continued pretraining**. In their experiments, they use a cosine decay schedule, starting from a "peak" LR (**on the order of 4.5e-5 in their setup**) and decaying toward a low minimum.

CONTINUAL PRETRAINING

Reference	What they say / why it's relevant
A Learning Rate Path Switching Training Paradigm for Version Updates of Large Language Models (Wang et al. 2024)	Proposes a "path-switching" scheme for updating large models when new data arrives: use a large LR in an initial **"rehydration / reinitialization"** stage, then fully decay the LR in the CPT stage. They find that **this two-stage strategy is important for balancing performance and training cost** when doing continual updates.

CONTINUAL PRETRAINING

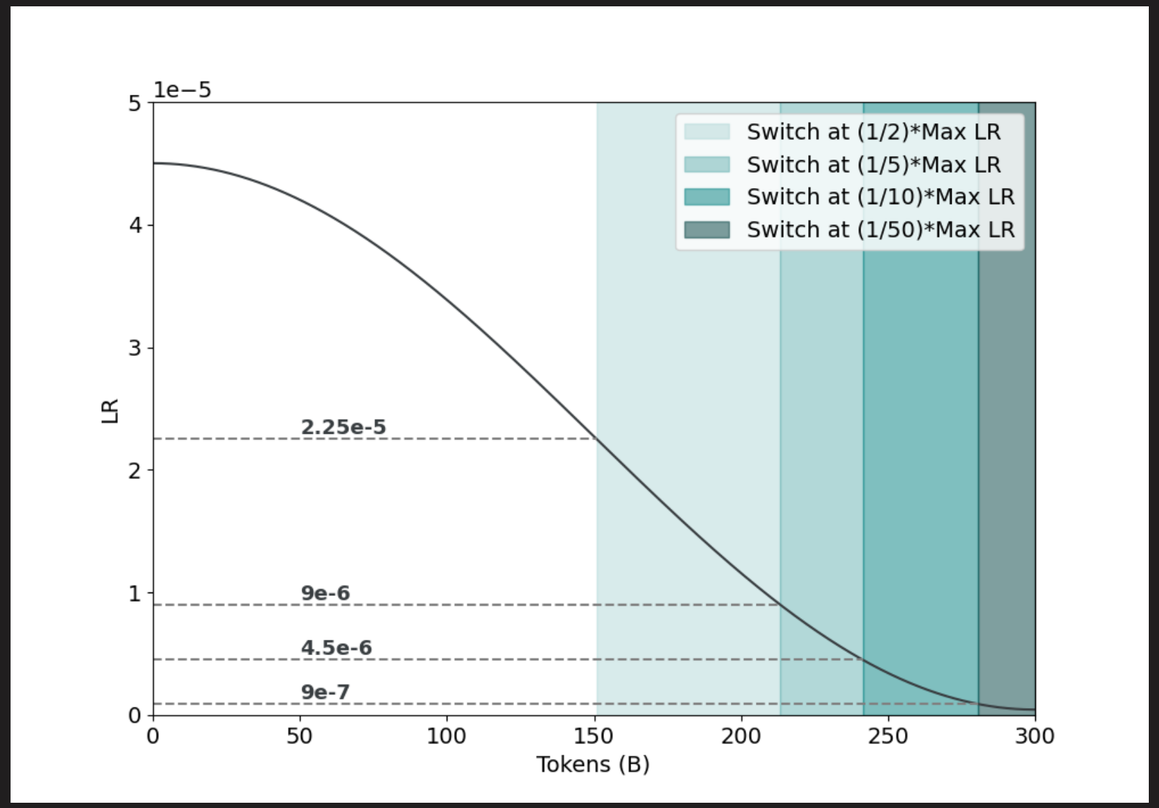
Reference	What they say / why it's relevant
Learning Dynamics in Continual Pre-Training for Large Language Models (Wang et al. 2025)	Introduces a scaling law for CPT: **they decouple the effect of distribution shift (old → new data) from learning rate annealing**, and offer a principled way to predict how loss will evolve under various LR schedules and hyperparameters. This allows estimating **an optimal "peak LR," number of CPT steps, replay ratio, etc., depending on your performance goals** (e.g., keep general knowledge vs specialize).

CONTINUAL PRETRAINING

Reference	What they say / why it's relevant
Revisiting Replay and Gradient Alignment for Continual Pre-Training of Large Language Models (Abbes et al. 2025)	This paper is about replay & gradient alignment to mitigate forgetting. It underscores that **learning rate interacts with data replay & stability**: choosing an appropriate LR schedule remains important even (or especially) when replaying old data, to avoid instability or catastrophic forgetting.

A "SIMPLE" RECIPE FOR CPT LEARNING RATE SCHEDULING

- ✓ **Start with a data distribution that is similar to the pretraining set** but places a larger weight on high-quality sources
- ✓ Transition to a **second distribution that incorporates your domain** of interest.
- ✓ The learning rate schedule should **start from min learning rate of the pretrained model** and decay with cosine annealing to min learning rate / 100.
- ✓ The **switch between data distribution should occur at min learning rate / 5** in the learning rate schedule.



Reuse, Don't Retrain: A Recipe for Continued Pretraining of Language Models

TOKENIZER TRANSPLANTATION

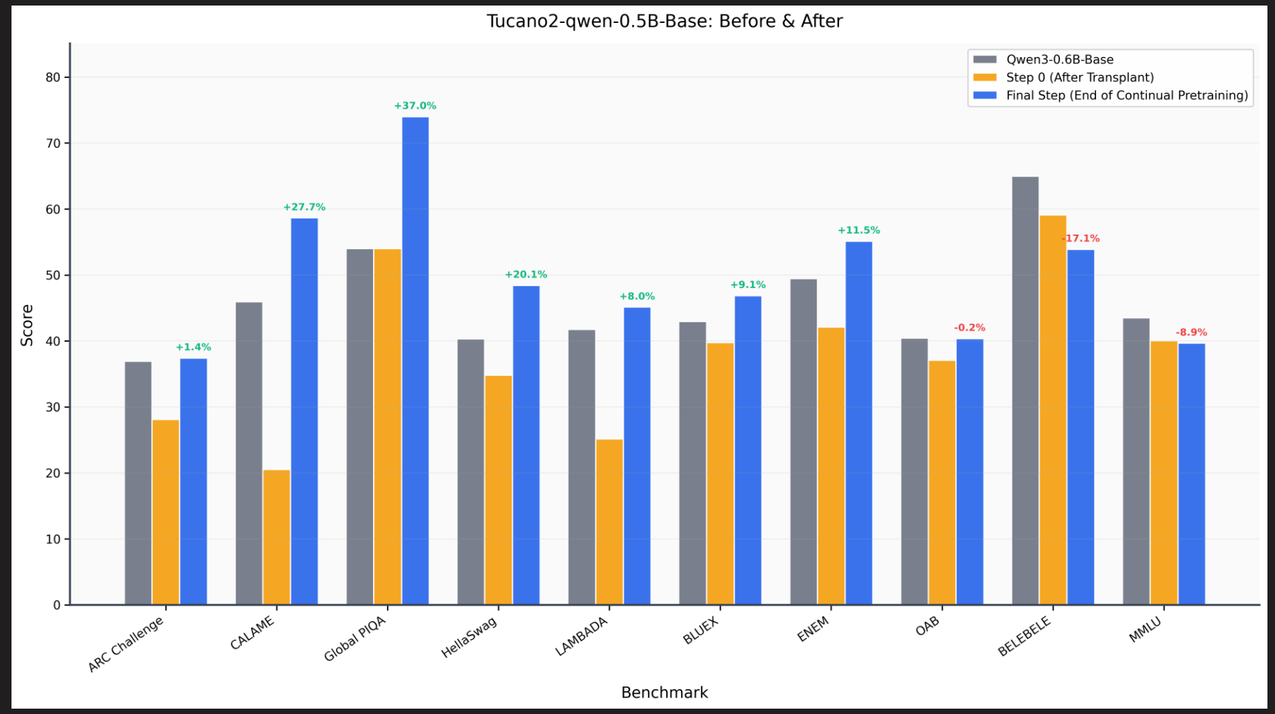
- ✓ Modern state-of-the-art multilingual LLMs (e.g., **Qwen, DeepSeek, Kimi**) employ extremely large vocabularies. Such large vocabularies are **expensive to train**, since the corresponding logits must be materialized during cross-entropy loss computation.

- ✓ If we are training an LLM for a low-resource language *X*, we may **not require the full set of tokens and embeddings that encode languages outside our target domain**.

- ✓ However, pruning the tokenizer or embedding layer can have unintended consequences and degrade performance. Because **representations** in neural networks are **highly distributed**, removing an embedding that appears irrelevant may still eliminate information that is implicitly shared with embeddings we care about. It is therefore **difficult to guarantee that pruning preserves all useful knowledge**.

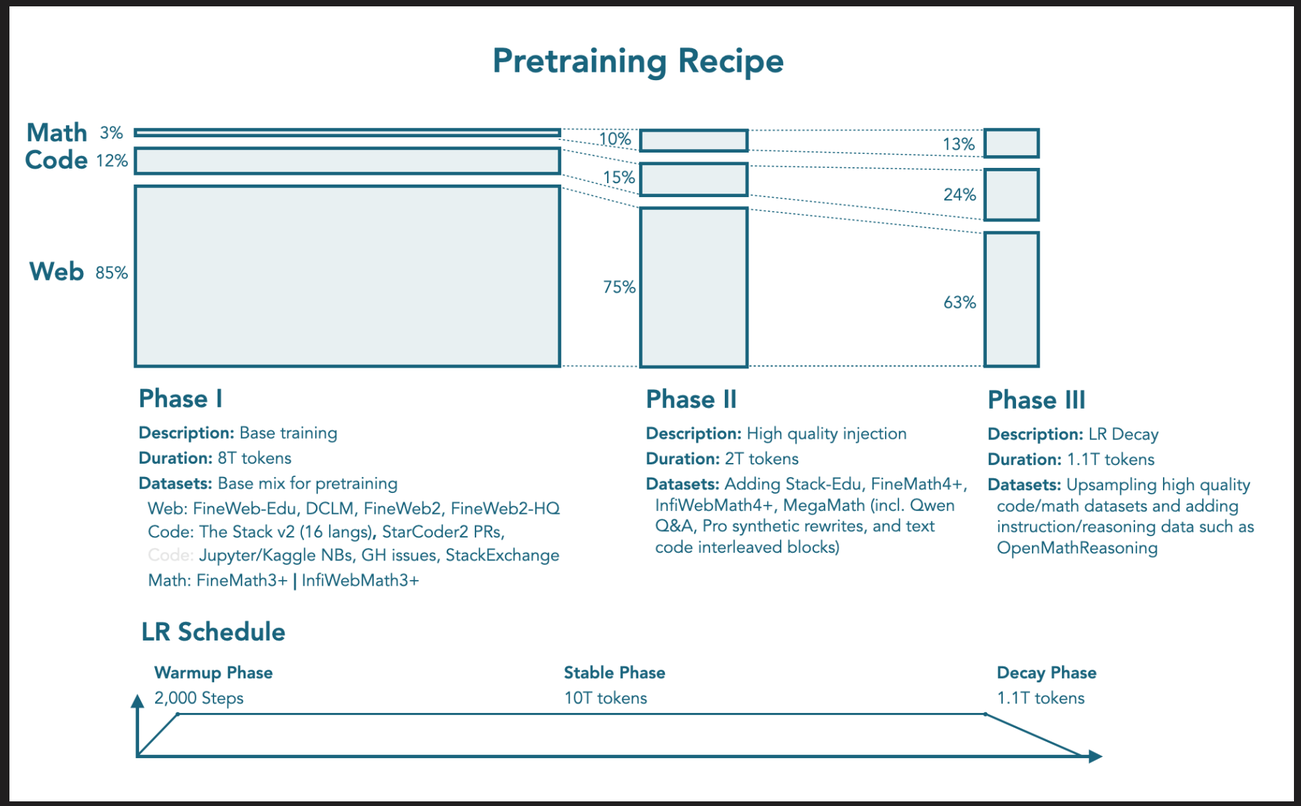
TOKENIZER TRANSPLANTATION

- ✓ Something different from pruning to try is ***transplantation***.
  Transplantation **reconstructs embeddings for a donor tokenizer inside the base model's embedding space so that the resulting model can operate with the donor vocabulary and ID mapping**.
- ✓ Orthogonal Matching Pursuit (OMP) tries to approximate unseen token embeddings as sparse combinations of tokens shared between the two vocabularies. This provides a **training-free way to align tokenizers** with minimal loss in downstream performance.



CURRICULUMS AND TRAINING RECIPES

Model(s)	Recipe
SmolLM2	
SmolLM3	
OLMo-2	
OLMo-3	
APERTUS	
LilMoo	
LilTii	
Tucano2	



Source → SmolLM3: smol, multilingual, long-context reasoner

CHECKPOINT MANAGEMENT

Training will **fail**. Jobs will **crash**. Interruptions are **inevitable in any long-running experiment**. Checkpoints are our safety net for these marathon training runs.

- ✔ Recovery from failures.
- ✔ Monitoring training progress via evaluation.
- ✔ Sharing intermediate models with the research community

Among these, recovery is the most critical. When a run fails, **we want to resume from the most recent checkpoint so that we lose, at worst, a single save interval**.

- 💡 **Automate your resume process whenever possible (#SBATCH --requeue).**
- 💡 **Set up alerts so you know when things go wrong (#SBATCH --mail-user, #SBATCH --mail-type)**

PRE-MARATHON CHECKLIST

Infrastructure readiness:

- ✔ **Reservations** avoid queueing delays, allow for quick debugging, and will keep you on a schedule.
- ✔ **Stress-test** GPUs before launch (e.g., DCGM Diagnostics) to catch performance degradation.

Evaluation setup:

- ✔ **Automate** everything you can.

Logging setup:

- ✔ Make sure you're logging all the metrics you care about.

FINAL THOUGHTS

Assume hardware is mortal.

✔ GPUs fail, networks wobble, and long runs stress systems in creative ways. Plan accordingly.

Patch fast, fix later.

✔ Under tight deadlines, proven workarounds often beat heroic debugging.

Know when to rewind vs. reboot.

✔ Loss spikes are the signal. Recoverable → rewind; Non-recoverable → restart and investigate.

Loss curves don’t catch everything.

✔ Run downstream evaluations early and often; silent bugs exist.

Innovate selectively.

✔ Track major advances, adopt those with strong empirical wins, ignore marginal hype.

FINAL THOUGHTS

Be systematic, not optimistic.
- ✔ Validate each change independently before stacking ideas.

Small-scale results don’t always scale.
- ✔ Re-ablate at target size when possible.

Spend compute where it matters most.
- ✔ Data quality and coverage usually beat minor algorithmic tweaks.

Prefer stability over fragile peaks.
- ✔ If performance is similar, choose the more robust and flexible method.

Stop tuning. Start training.
- ✔ The finished model beats the perfect model that never ran.

VALUABLE LORE

A detailed log that captures real **struggles**, **panics**, and the steps taken to resolve potentially **catastrophic bugs** is incredibly **instructive and valuable**. Such logs provide insight into complex failure modes, document critical decision-making, and serve as an essential resource for reproducibility and future contributors. For any **serious project, logging your development work should be mandatory, not optional**.

- ✔ *Training LLMs on HPC Systems: Best Practices from the OpenGPT-X Project*, by **Penke et al.**
- ✔ *The Ultra-Scale Playbook: Training LLMs on GPU Clusters*, by **Tazi et al.**
- ✔ *Training Chronicles of the 176B-parameter multilingual LLM*, by **Stas Bekman.**

TRAINING LOOP AND CHECKPOINTING

- ✔ Learn how to implement the main training loop with proper epoch and iteration management.
- ✔ Learn how to handle gradient accumulation with DDP synchronization.
- ✔ Learn how to perform forward and backward passes with mixed precision using autocast.
- ✔ Learn how to implement gradient clipping and optimizer steps with dynamic learning rates.

EXERCISE

BREAK → EVALUATIONS